Final Project Report

Design and Implementation of a 4-Bit Very Simple Microprocessor with UART Communication

Terasic DE10-Lite FPGA | Verilog HDL

Student: Monowar Mahmud

Course: ENCE4110

1 Project Overview

This project implements a 4-bit Very Simple Microprocessor (VSM) in Verilog HDL on Terasic DE10-Lite FPGA. It consists of control unit, datapath, arithmetic unit, program memory and input/output interfaces. The processor has a 4-bit bus and 8-bit instructions. It supports six operations: NOP, LDA, ADD, SUB, IN and OUT. A UART interface allows the user to send and receive hexadecimal data to/from a computer. A reduced processor clock allows the user to monitor instruction execution via onboard displays . The report outlines the design and operation of the implemented modules.

2 Accumulator A

Purpose and implementation

The main working register of four bits is the `Accumulator_A` module. It contains the value being processed by the VSM, which can be immediate operands, input data, arithmetic results etc. The register is parameterized with `N` with the top-level design having $N = 4$.

- `MainClock` is the clock for the register; `ClearA` synchronously clears the stored value.
- `LatchA` allows the value currently on `IB_BUS` to be loaded on a rising clock edge.
- `AluA` drives the stored accumulator value to the arithmetic unit at all times.
- `EnableA` allows accumulator A to drive the shared internal bus, otherwise its bus driver produces high impedance in the RTL description.

During LDA and IN, the controller selects the instruction operand or external input and asserts `LatchA`. In ADD and SUB the ALU result is written back to the accumulator A. During OUT, the accumulator A presents the value stored in the output register. So accumulator A is the primary storage for the processor's working value between instructions.

3 Accumulator B

Purpose and implementation

The `Accumulator_B` module is another 4-bit clocked register that has an arithmetic operand. It accepts data from the shared internal bus and makes the stored value available through `AluB`.

- `LatchB` allows to load the bus value on a rising edge of `MainClock`.
- `ClearB` synchronously clears the register.
- `AluB` is wired to the B input of the arithmetic unit.

Accumulator B does not drive the internal bus in this implementation, unlike accumulator A. For an ADD or SUB instruction, the instruction register places its immediate operand on the bus during the relevant execution phase and accumulator B captures that operand. The ALU can then combine the previously stored value of accumulator A with the newly captured value of B. The registers are separated so that the original value of the accumulator A is not overwritten before the arithmetic calculation.

4 Arithmetic Logic Unit (ALU)

The `Arithmetic_Unit` module is a combinational arithmetic block which performs four-bit addition as well as subtraction . `AluA` and `AluB` are read from the two accumulators. The operation is selected by the `AddSub` input:

$$R = \begin{cases} A + B, & \texttt{AddSub} = 0, \\ B - A, & \texttt{AddSub} = 1. \end{cases}$$

The RTL computes the result as five bit intermediate signal `sum`. If `EnableALU` is asserted, it puts the lower four bits on `IB_ALU`. Otherwise the ALU bus driver is high impedance in RTL. The fifth bit is exported as `Carry`. In subtraction, it corresponds to the upper bit of the unsigned extended difference, and can be used to detect underflow. The four bit output can be naturally wrapped modulo 16. For example $15+3 = 18$ and last 4 bits of 18 is 2. For the programmed subtraction, accumulator B contains 8 and accumulator A contains 7, so $B - A = 8 - 7 = 1$. The ALU itself does not store the results . The FSM asserts `LatchA` , so the result is captured by accumulator A at the clock edge .

5 Input Register

The `InRegister` module connects a selected four-bit external value to the internal bus of the VSM. Its RTL is combinational. When `EnableIN` is high, `DataIn` is driven onto `IB_BUS`. The bus driver is high impedance otherwise. Although the name suggests otherwise, there is no clocked data storage element in this block. When the FSM executes the IN instruction and asserts `LatchA`, the actual input value is written into accumulator A. At the highest level `SW[7]` selects the input source:

$$\texttt{w_user_input} = \begin{cases} \texttt{SW[3:0]}, & \texttt{SW[7]} = 0, \\ \texttt{w_uart_input}, & \texttt{SW[7]} = 1. \end{cases} \tag{1}$$

This can be achieved either via manual hex input from the FPGA switches or via the UART interface for input from the computer. The selected value is also displayed on `HEX4`.

6 Output Register

The `OutRegister` module stores the last four-bit value generated by an OUT instruction. When `MainReset` is asserted it synchronously clears `rOut` on a rising edge of `MainClock`. Otherwise if `EnableOut` is high, it saves the current `IB_BUS` value. For an OUT instruction, the FSM asserts `EnableA` and `EnableOut`. Accumulator A drives the internal bus and the output register latches the value. The output is stored and held until the next OUT instruction or reset. The top-level design routes `rOut` to `LEDR[3:0]`, `HEX5`, and the UART interface. This lets the same processor result be seen locally and sent to the computer.

7 Instruction Register

The `InstructionReg` module receives an eight-bit instruction from ROM as two four-bit signals: `Instr` (opcode) and `Data` (operand). It also saves when `LatchInstr` is high at the rising edge of `MainClock`. The `ToInstr` signal feeds the stored opcode into the FSM, while the operand is stored in the internal

DataReg register.

$$\underbrace{\texttt{Instruction[7:4]}}_{\text{opcode}} \quad \underbrace{\texttt{Instruction[3:0]}}_{\text{operand}}. \tag{2}$$

When EnableInstr is asserted, the stored operand drives the shared bus. This is needed for LDA and loading accumulator B during arithmetic instructions. The register also has a synchronous clear input, ClearInstr. Splitting opcode and operand enables the FSM to decode an instruction while its immediate data is available to the datapath.

8 Program Memory (ROM)

The ROM_Nx8 module implements eight instruction locations of eight bits each. The memory is initialized in the Verilog source, and a combinational read returns rom[address]. The top level design connects the lower three bits of the program counter to the ROM address input.

Address	Hex	Opcode	Operand	Intended operation
0	55	5	5	LDA 5
1	12	1	2	ADD 2
2	30	3	0	OUT
3	28	2	8	SUB 8
4	30	3	0	OUT
5	40	4	0	IN
6	13	1	3	ADD 3
7	30	3	0	OUT

The first three instructions load 5 , add 2 , and output 7 . The next two subtract the value stored in the accumulator from 8 and output 1 . The last 3 instructions are read in an input , add 3 , and output . The ROM uses the three least significant bits of the program counter as its address and the instruction sequence repeats.

9 Program Counter

The ProgramCounter module holds the instruction pointer. It is implemented as a four-bit clocked counter with inputs EnableCount and synchronous ClearCounter. When enabled, the counter counts on the rising edge of the clock. In its decode phase, the FSM asserts EnableCount. Only Counter[2:0] addresses the eight-word ROM, so the effective ROM address goes as $0, 1, \ldots, 7, 0, 1, \ldots$. The full value of the four-bit counter is shown on LEDR[7:4] So the counter provides for instruction sequencing and a visible indication of where the program is at the moment. The ROM address wraps every eight counts, but its four-bit value may pass through 8–15.

10 Finite-State Machine (FSM) Controller

The module FSM_MicroInstr has five states IDLE, PHASE_1, PHASE_2, PHASE_3 and PHASE_4 and orchestrates the datapath. State transitions are clocked. The control outputs are generated from combinational logic based on the state and opcode. The control outputs default low and are asserted only for the relevant operation.

State	Main function
IDLE	Initial state after reset, then advance to instruction fetch.
PHASE_1	Assert `LoadInstr` to capture the ROM instruction.
PHASE_2	Assert `EnableCount` and `EnableInstr`; advance the program counter and make the operand available on the bus.
PHASE_3	Execute instruction-dependent transfer: load B for ADD/SUB, load A for IN/LDA, or capture A in the output register for OUT.
PHASE_4	For ADD/SUB, enable the ALU and latch the result into accumulator A; otherwise complete the instruction cycle.

Implemented instruction behavior

- **NOP (opcode 0):** No datapath update is requested.
- **ADD (opcode 1):** Capture immediate operand in B, compute $A + B$, and store the lower four bits in A.
- **SUB (opcode 2):** Capture immediate operand in B, compute $B - A$, and store the lower four bits in A.
- **OUT (opcode 3):** Enable A onto the bus and capture its value in the output register.
- **IN (opcode 4):** Enable the selected external input onto the bus and capture it in A.
- **LDA (opcode 5):** Enable the instruction operand onto the bus and capture it in A.

The controller uses the sequence `IDLE` → `PHASE_1` → `PHASE_2` → `PHASE_3` → `PHASE_4` → `PHASE_1`. This separates instruction fetch, operand transfer, and execution. The FSM RTL is retained without modification.

11 Integrated architecture

The `main` module instantiates the ROM, program counter, instruction register, FSM, two accumulators, ALU, input and output modules, display decoders, clock divider and UART interface. Several modules share the four-bit `w_IB_BUS` and the controller is designed to permit only one source at a time. The 7-segment displays show the internal bus (HEX0), accumulator A (HEX1), accumulator B (HEX2), selected input (HEX4) and output register (HEX5). The LEDs show the carry bit, UART selection switch, program counter, and output value.

11.1 Clocking and reset

The board supplies $f_{\text{clk}} = 50\,\text{MHz}$. The uploaded four-second divider produces one narrow slow-clock pulse for every 200,000,000 input cycles:

$$T_{\text{slow}} = \frac{200{,}000{,}000}{50 \times 10^6} = 4\ \text{s}. \tag{3}$$

This slows down the phases of the instructions for visual inspection. The top-level design also combines a registered reset-related signal with the slow clock to produce reset clock edges for synchronous processor registers. As this is a logic implemented derived clock, it must be carefully checked in timing analysis and hardware testing.

11.2 Step-by-step program example

Assume the external input is hexadecimal `6`. The programmed instruction sequence is expected to perform the following operations:

Address	Instruction	Working value A	Output register
0	LDA 5	5	unchanged
1	ADD 2	7	unchanged
2	OUT	7	7
3	SUB 8	1	unchanged
4	OUT	1	1
5	IN	6	unchanged
6	ADD 3	9	unchanged
7	OUT	9	9

Therefore the expected OUT values are $7 \rightarrow 1 \rightarrow 9$ and repeat on subsequent ROM cycles if the input does not change. If the input is hexadecimal A (decimal 10), the third result is hexadecimal D (decimal 13).

12 UART Integration (Graduate Students)

The UART subsystem expands the VSM with serial communication between the DE10-Lite and a computer. The uploaded `main.v` maps `GPIO[33]` to UART receive and `GPIO[35]` to UART transmit. A compatible 3.3 V TTL USB–UART adapter should connect the TXD to the FPGA RX, the RXD to the FPGA TX and its ground to the FPGA ground. These HDL GPIO indices must match the intended physical header pins in the Quartus pin assignments.

12.1 UART receiver

The `async_receiver` module samples incoming serial characters at 115200 baud from the 50 MHz clock. It uses 2 flip-flops to synchronize the asynchronous input . It uses IDLE , START , DATA , and STOP states . It receives 8 data bits , with no parity and one stop bit . The receiver samples in the middle of each bit and generates a one-clock `RxD_data_ready` pulse after a valid stop bit. The rewritten implementation does direct counting of the clock instead of an oversampling filter.

12.2 ASCII-to-hexadecimal conversion

Each byte is received by the `VSM_UART_Interface`, which accepts ASCII characters `0-9`, `A-F`, and `a-f`. Takes a valid character and stores it as a four bit value in `uart_input`. Invalid characters are not replaced by the previously stored input. When `SW[7]` is asserted, the value stored is fed as the VSM input for the next IN instruction.

12.3 UART transmitter

The `async_transmitter` module sends 8-bit characters serially at 115200 baud, 1 start bit, 8 data bits, no parity, and 2 stop bits. It uses `BaudTickGen` to generate the transmission timing and asserts `TxD_busy` during transmission.

12.4 VSM output transmission

The UART interface detects the falling edge of `vsm_enable_out` by synchronizing registers and treats it as the completion indication of an OUT operation. It translates the present 4 bit output to an ASCII hex digit and requests transmission. This request is coordinated with the busy indication of the transmitter by a small TX control FSM. Hence, the expected output sequence for an input of 6 is seen as the characters `719719...` in a serial terminal, with no spaces or newline characters inserted.

13 Conclusion

The RTL design is a four-bit processor with on-board visualization and instruction-controlled UART communication. The datapath has two accumulators and a combinational ALU . The FSM controls register loading and shared-bus transfers . The ROM contains an eight-word repeating demonstration program to exercise load, arithmetic, input and output instructions. The UART extension offers the same four bit input/output functions to communicate with a computer via ASCII hexadecimal characters.

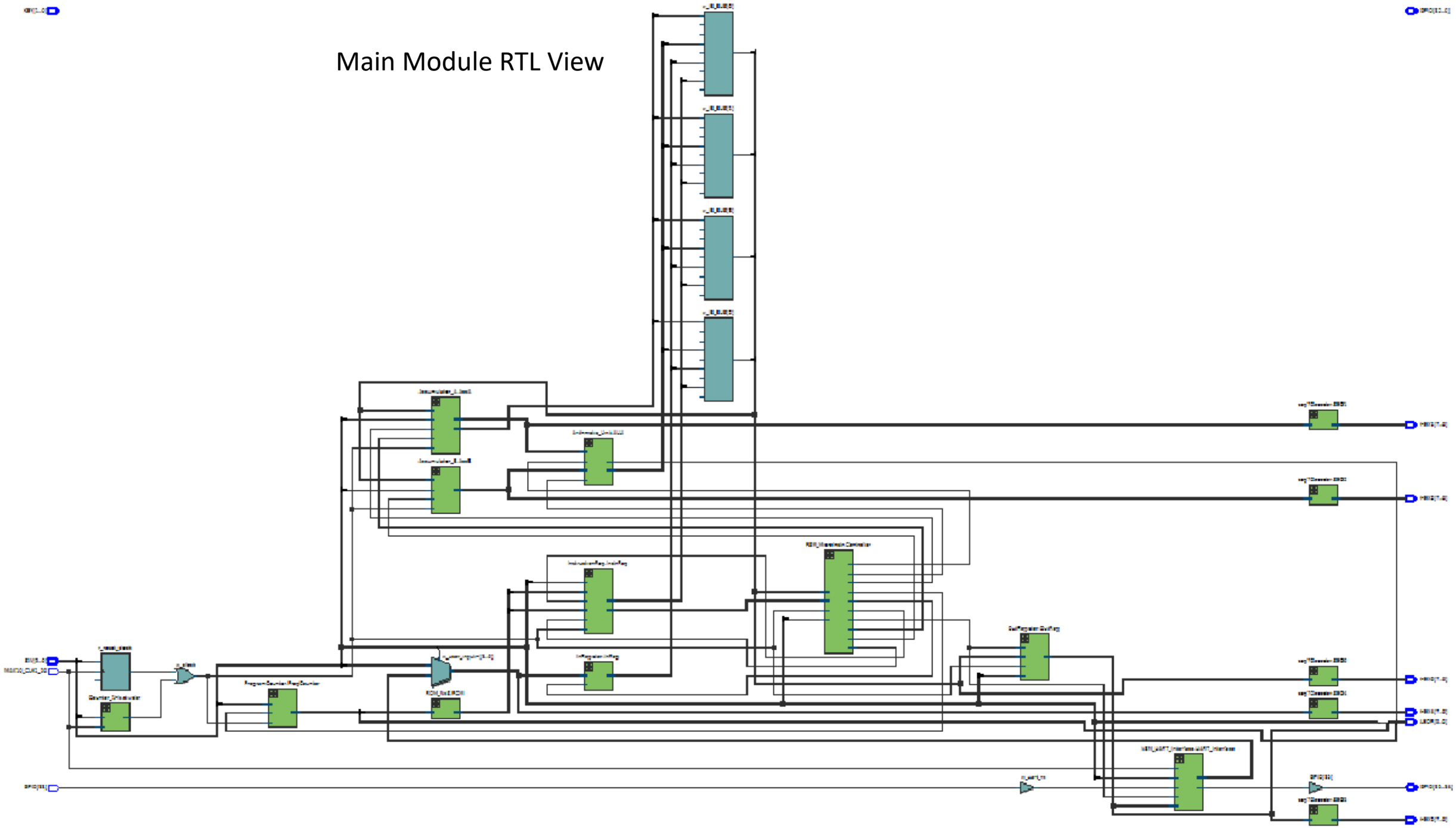
Project Overview

Design and Implementation of a 4-Bit Very Simple Microprocessor (VSM) with UART Communication

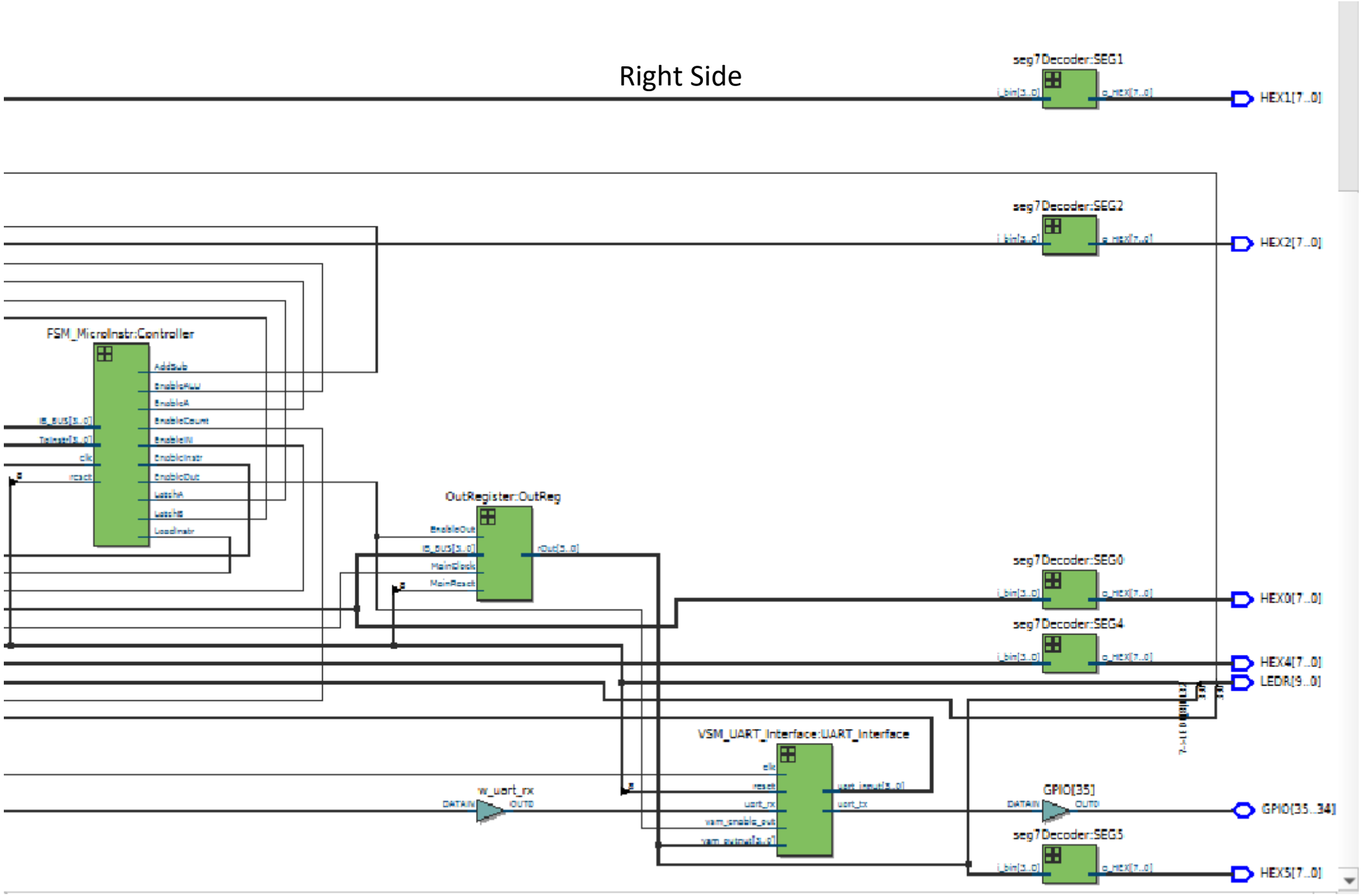
Top-Level Module

- Integrates all processor, display, clock, and UART modules.
- Connects Datapath components through a shared 4-bit internal bus.
- Distributes FSM-generated control signals to the processor modules.
- Uses SW[7] to select between manual and UART input.
- Connects UART RX and TX through FPGA GPIO pins.
- Displays internal processor values using LEDs and seven-segment displays.

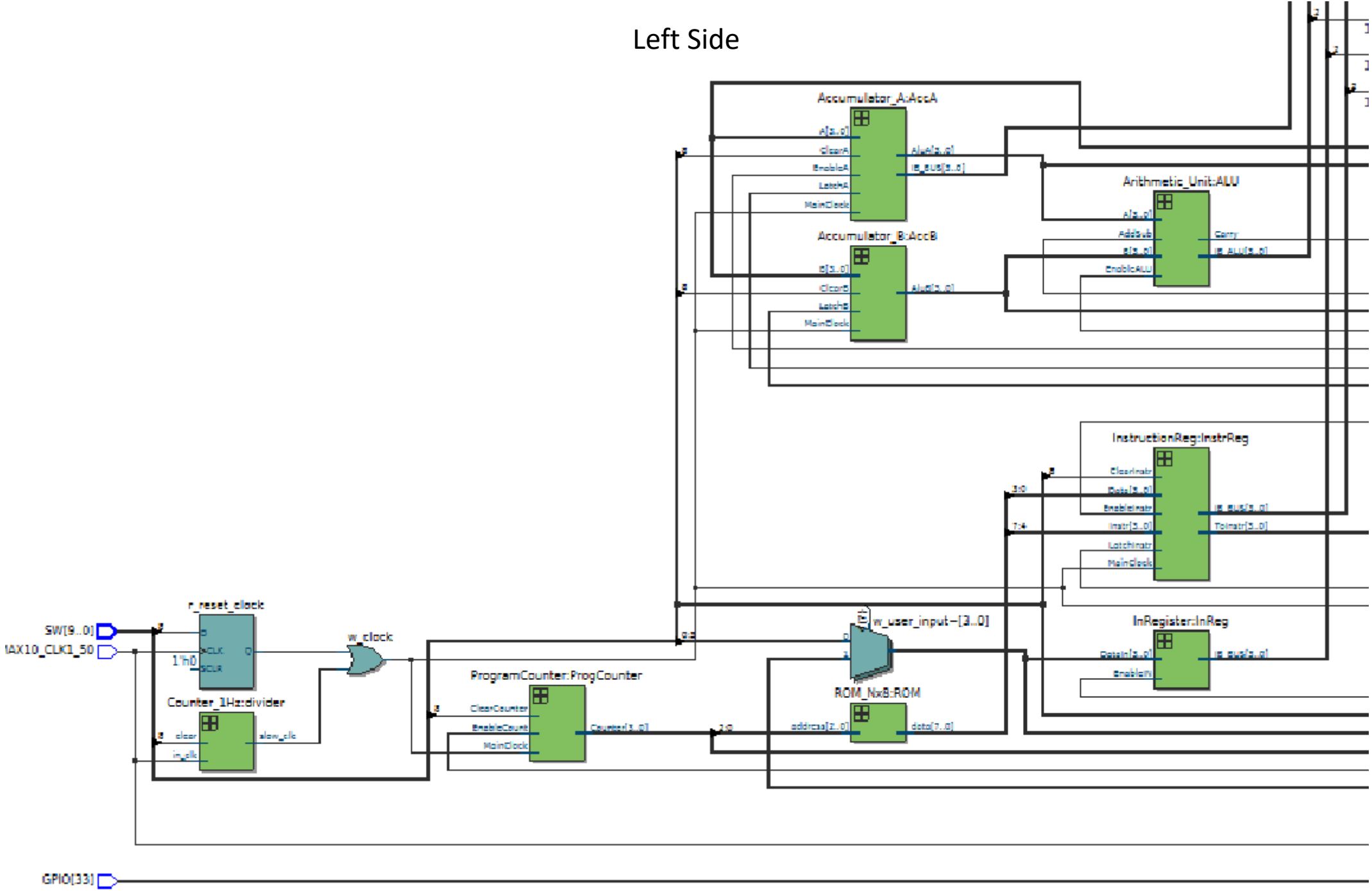
Main Module RTL View



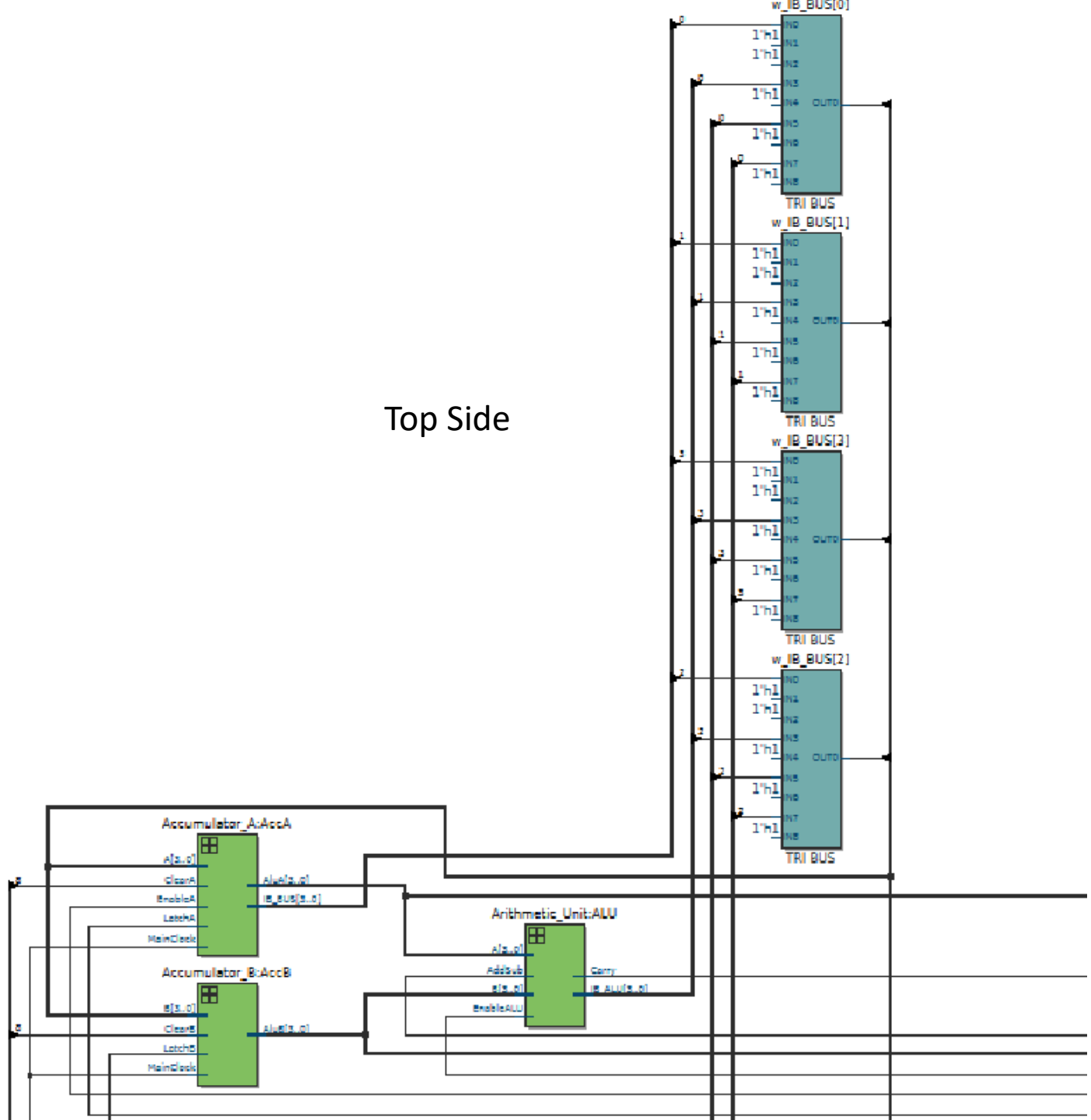
Right Side


seg7Decoder:SEG1
i_bin[3..0]
o_ntx[7..0]
HEX1[7..0]
seg7Decoder:SEG2
HEX2[7..0]
FSM_MicroInstr:Controller
AddSub
EnableRW
EnableA
EnableCount
EnableIW
EnableInstr
EnableOut
WatchA
WatchB
LoadInstr
IB_BUS[3..0]
clk
reset
OutRegister:OutReg
EnableOut
IB_BUS[3..0]
MainClock
MainReset
nOut[3..0]
seg7Decoder:SEG0
HEX0[7..0]
seg7Decoder:SEG4
HEX4[7..0]
LEDR[9..0]
VSM_UART_Interface:UART_Interface
clk
reset
uart_rx
vsm_enable_out
uart_tx
w_uart_rx
DATAIN
OUTD
GPIO[35]
GPIO[35..34]
seg7Decoder:SEG5
HEX5[7..0]

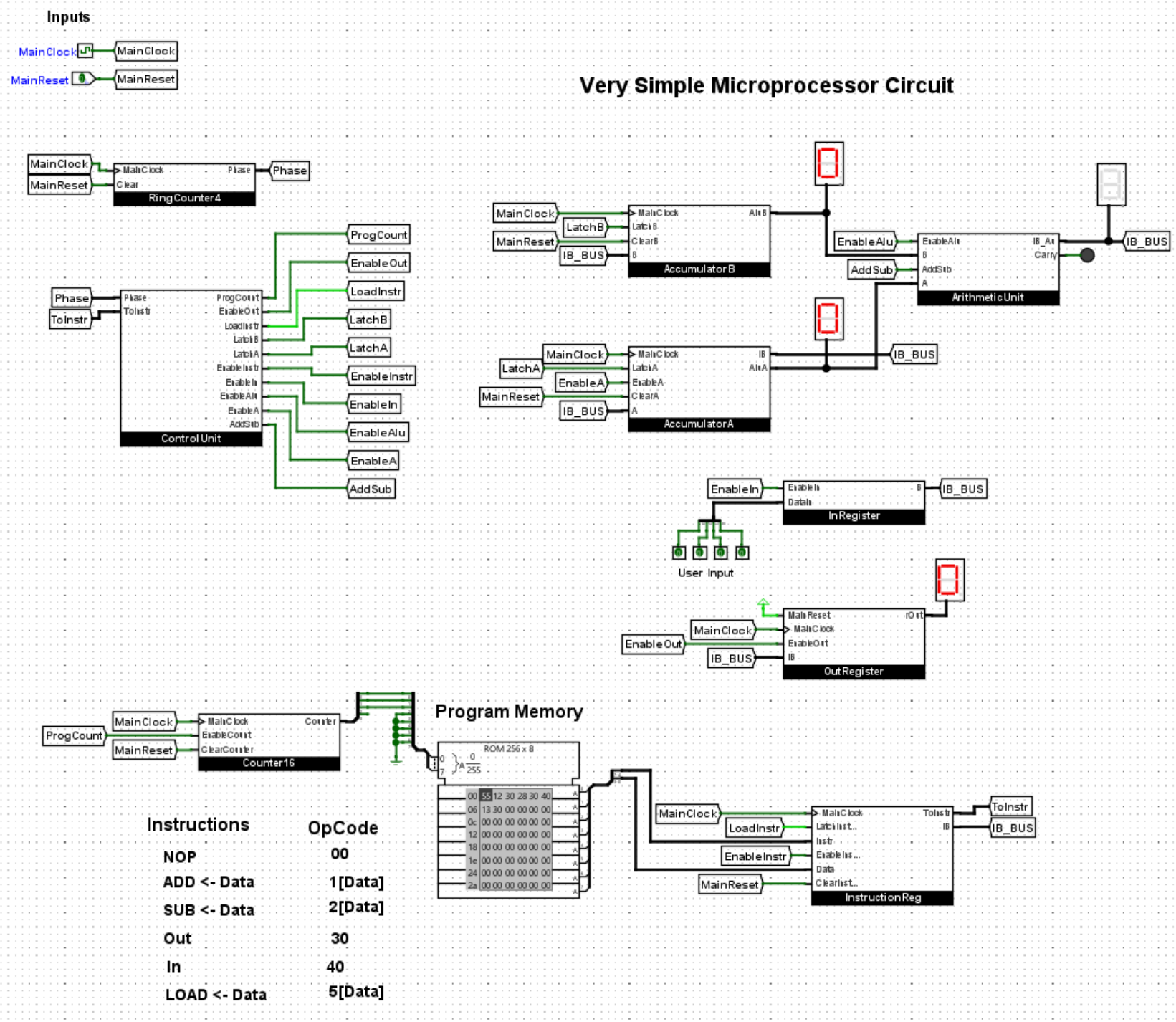
Left Side


Accumulator_A:AccA
Accumulator_B:AccB
Arithmetic_Unit:ALU
InstructionReg:InstrReg
InRegister:InReg
ROM_Nx8:ROM
ProgramCounter:ProgCounter
Counter_1Hz:divider
r_reset_clock
w_clock
w_user_input[3..0]
SW[9..0]
MAX10_CLK1_50
GPIO[33]

Top Side


w_IB_BUS[0]
w_IB_BUS[1]
w_IB_BUS[3]
w_IB_BUS[2]
TRI BUS
Accumulator_A:AccA
Accumulator_B:AccB
Arithmetic_Unit:ALU

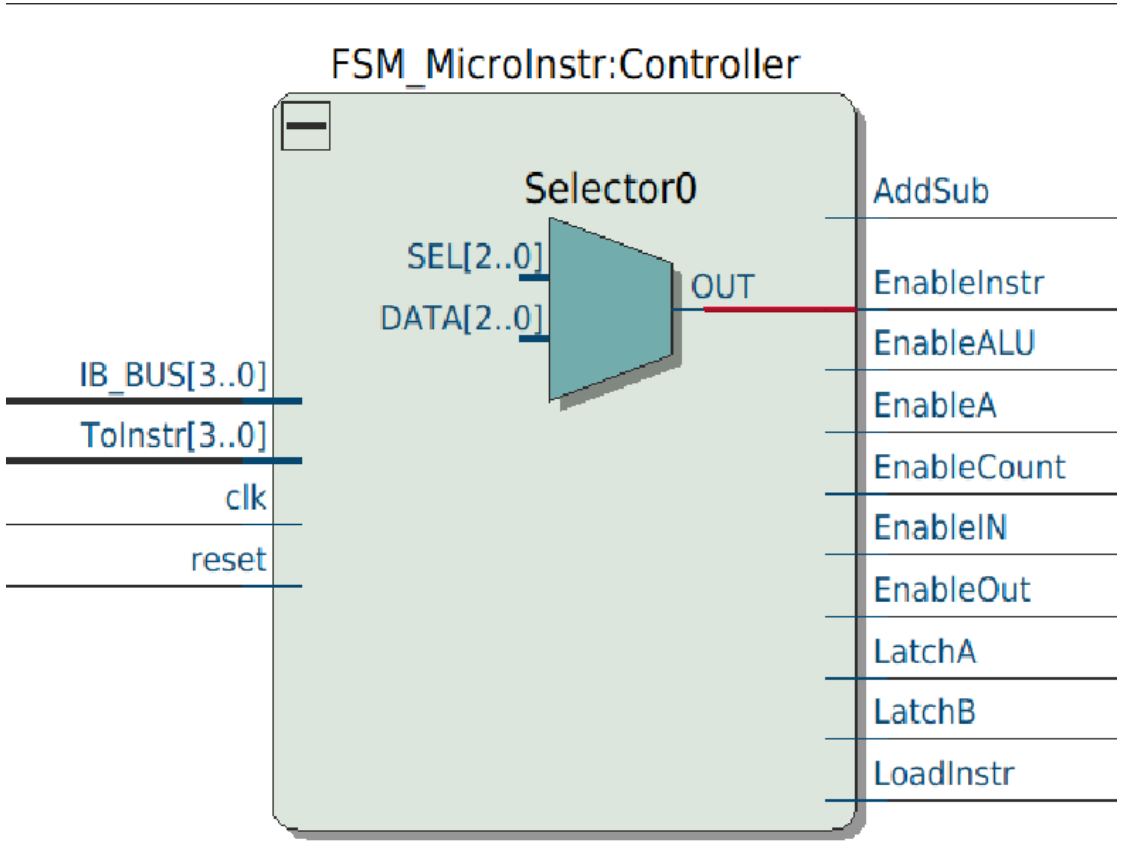
Logisim
Implementation



Microinstruction Controller (FSM_MicroInstr.v)

- Implements the processor control unit using a finite-state machine (FSM).
- Contains five states: IDLE, PHASE_1, PHASE_2, PHASE_3, and PHASE_4.
- Controls instruction fetching, decoding, and execution.
- Generates register-enable, latch, and arithmetic control signals.
- Supports NOP, LDA, ADD, SUB, IN, and OUT instructions.
- Coordinates data transfers across the shared internal bus.

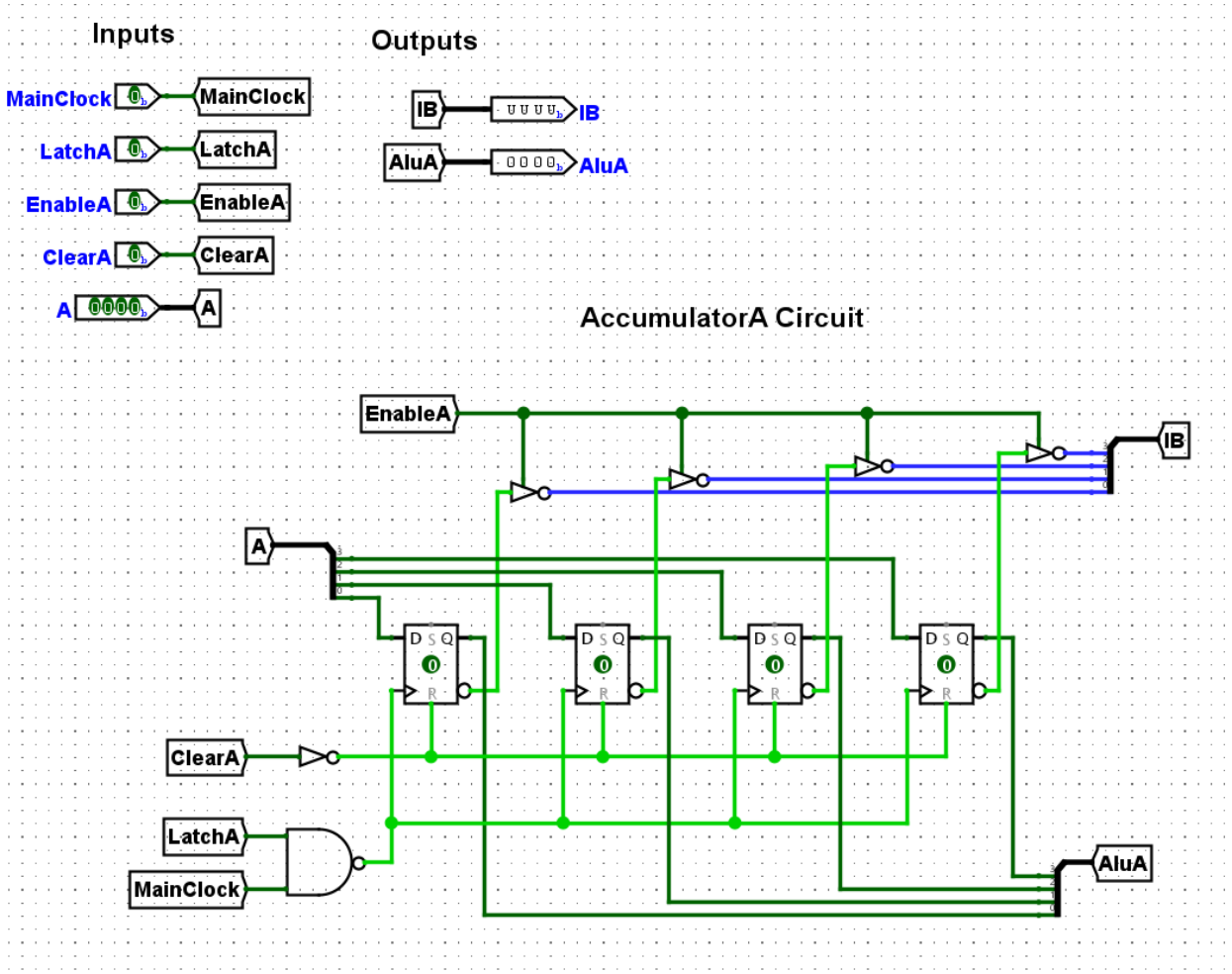
RTL View


FSM_MicroInstr:Controller
Selector0
SEL[2..0]
DATA[2..0]
OUT
IB_BUS[3..0]
ToInstr[3..0]
clk
reset
AddSub
EnableInstr
EnableALU
EnableA
EnableCount
EnableIN
EnableOut
LatchA
LatchB
LoadInstr

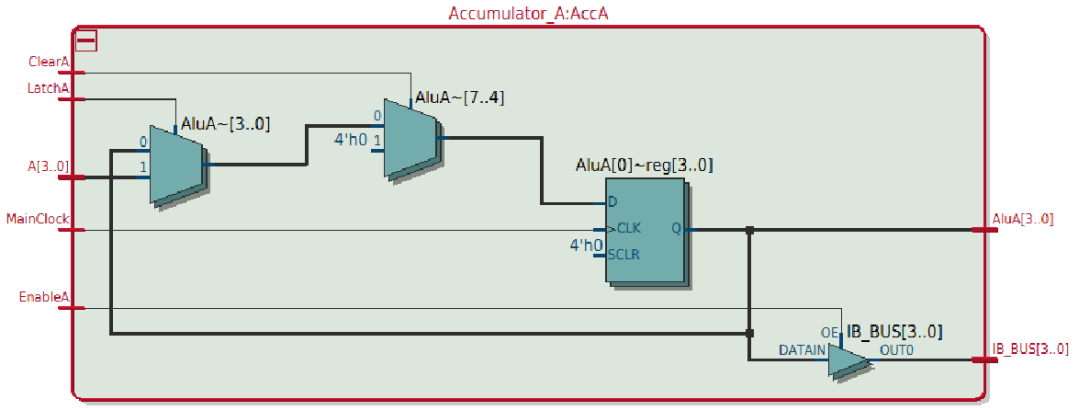
Accumulator A (Accumulator_A.v)

- Serves as the primary 4-bit working register.
- Stores operands and arithmetic results.
- Captures data from the internal bus when LatchA is enabled.
- Supplies operand A to the arithmetic unit.
- Drives the internal bus when EnableA is asserted.

Logisim Implementation



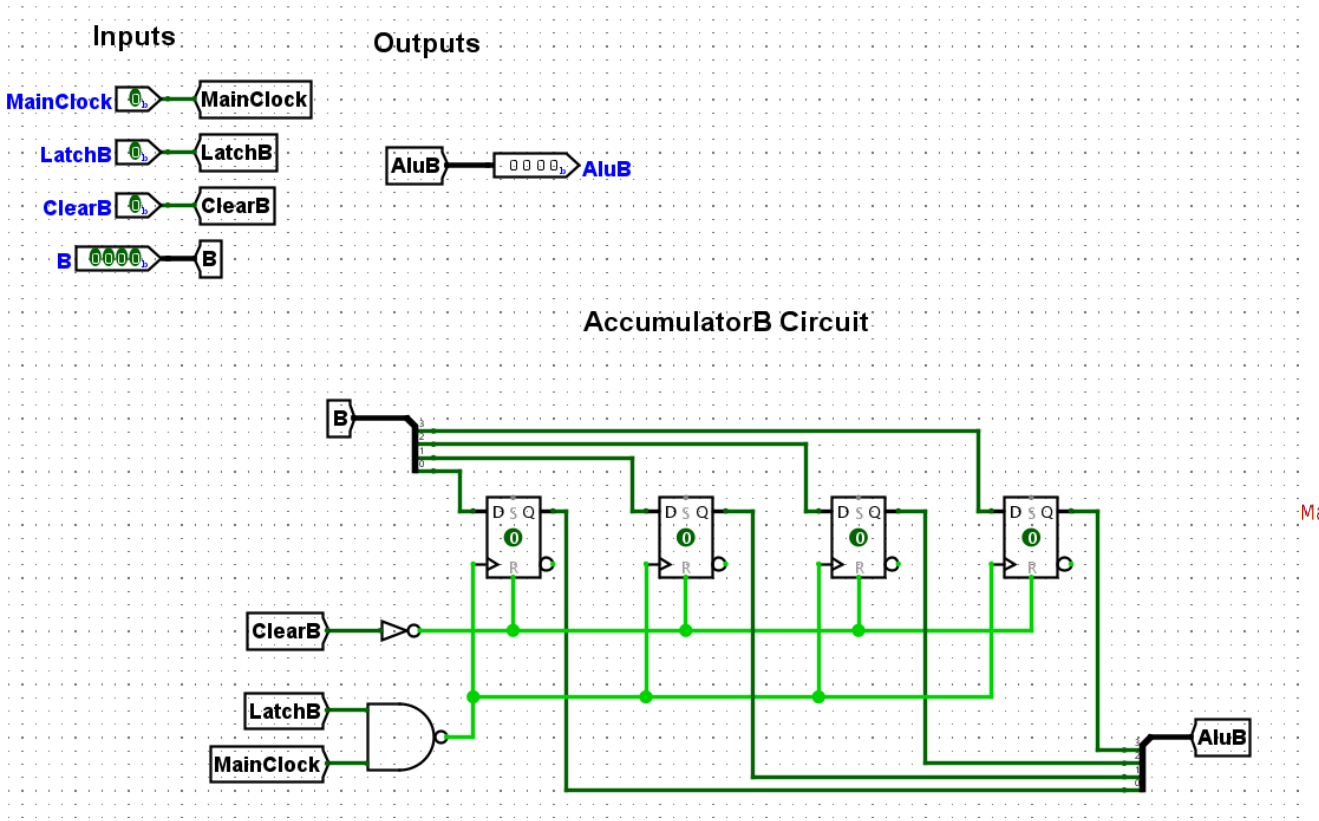
RTL View



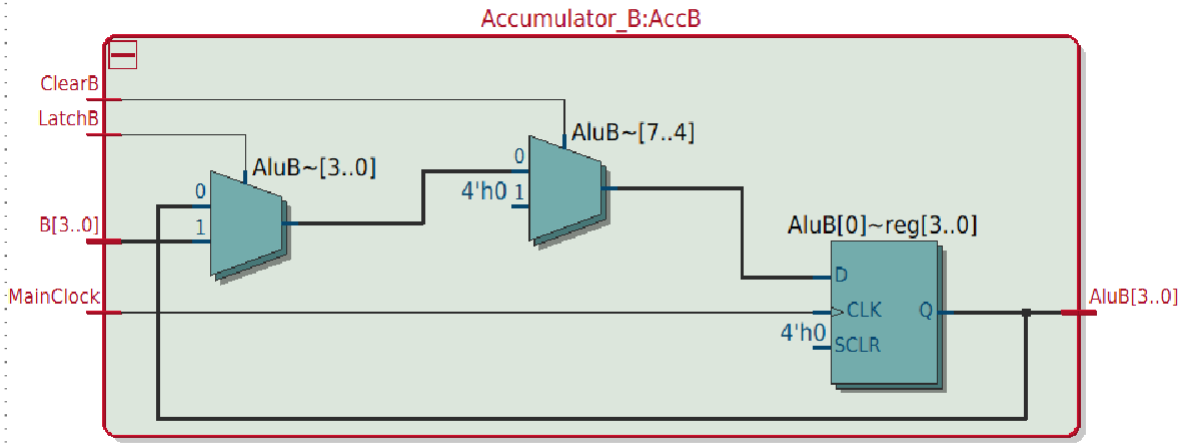
Accumulator B (Accumulator_B.v)

- Serves as a secondary 4-bit operand register.
- Stores the operand required for arithmetic operations.
- Captures data from the internal bus when LatchB is enabled.
- Supplies operand B directly to the arithmetic unit.
- Supports synchronous reset.

Logisim Implementation



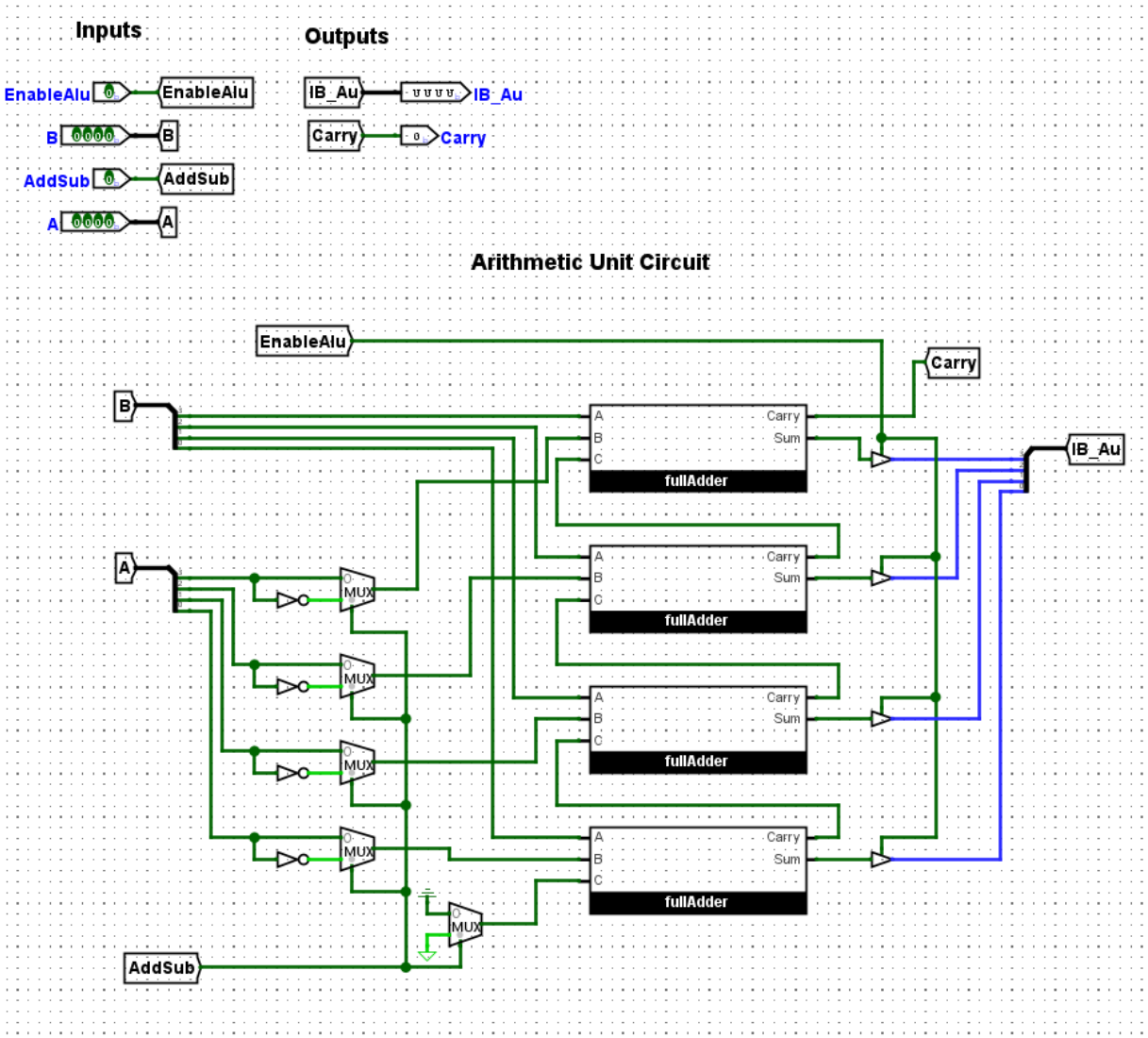
RTL View



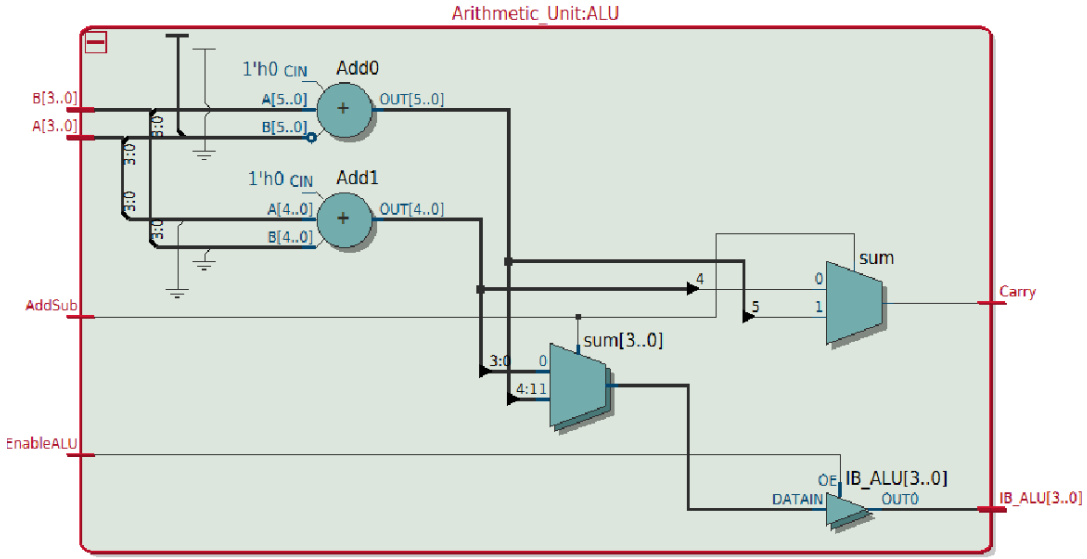
Arithmetic Unit (Arithmetic_Unit.v)

- Performs 4-bit binary addition and subtraction.
- Receives operands from accumulators A and B.
- Uses AddSub to select the arithmetic operation.
- Performs addition as A + B and subtraction as B − A.
- Uses a 5-bit intermediate result to retain carry or borrow information.
- Places the lower four result bits on the internal bus when enabled.

Logisim Implementation



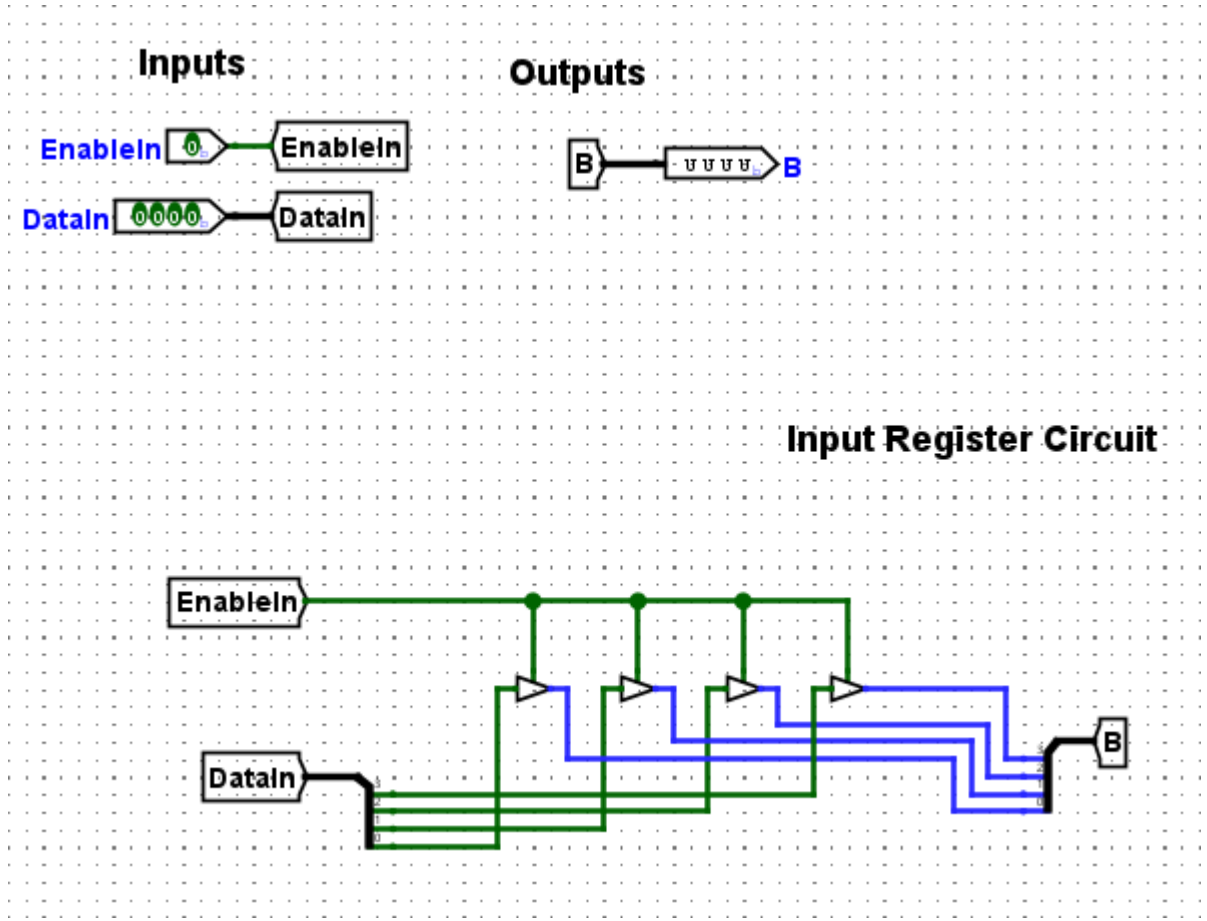
RTL View



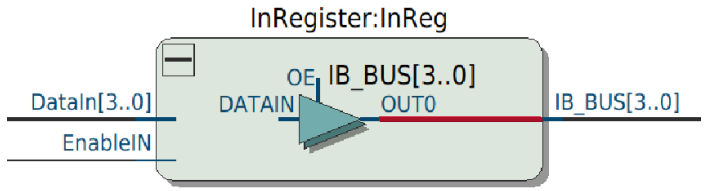
Input Register (InRegister.v)

- Connects external 4-bit data to the processor's internal bus.
- Accepts input from physical switches or UART.
- Transfers the selected input when EnableIN is asserted.
- Uses combinational bus-driving logic without internal data storage.

Logisim Implementation



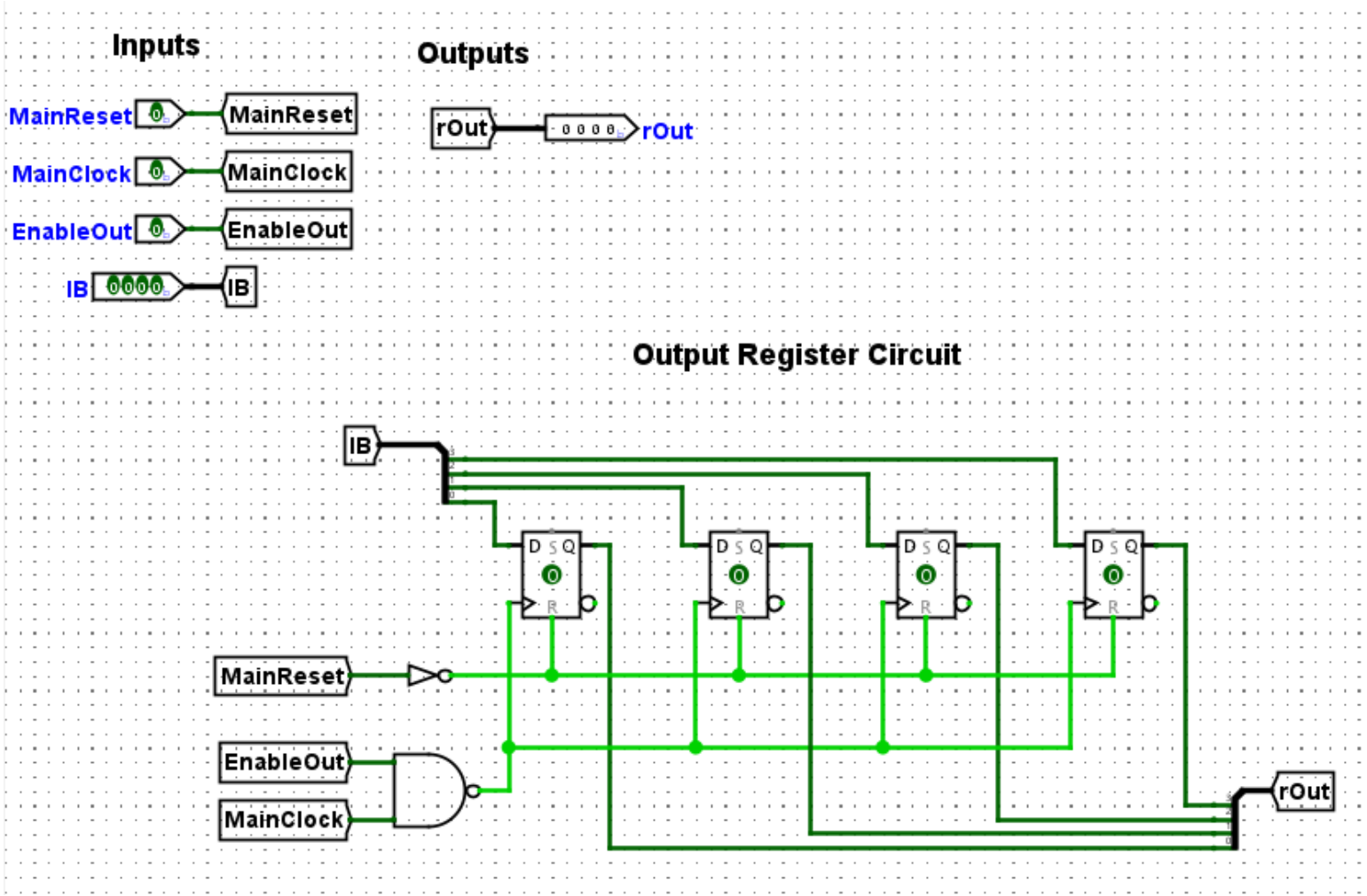
RTL View



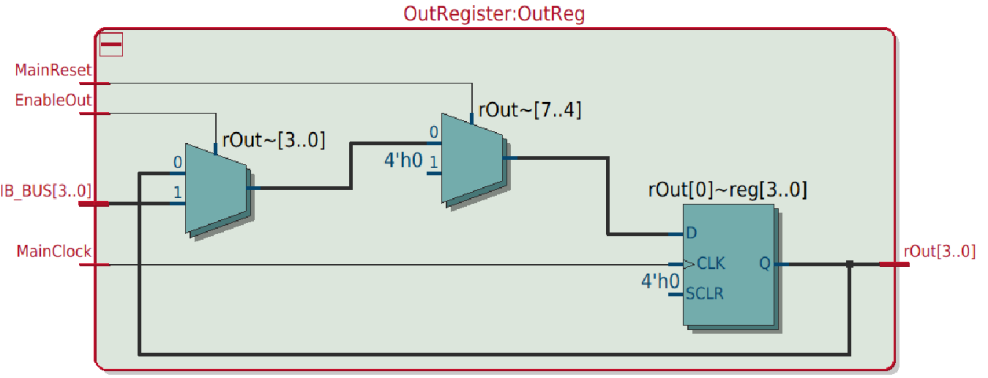
Output Register (OutRegister.v)

- Stores the processor's 4-bit output value.
- Captures internal bus data when EnableOut is asserted.
- Provides output to HEX5 and LEDR[3:0].
- Supplies the stored value to the UART interface for transmission.

Logisim Implementation

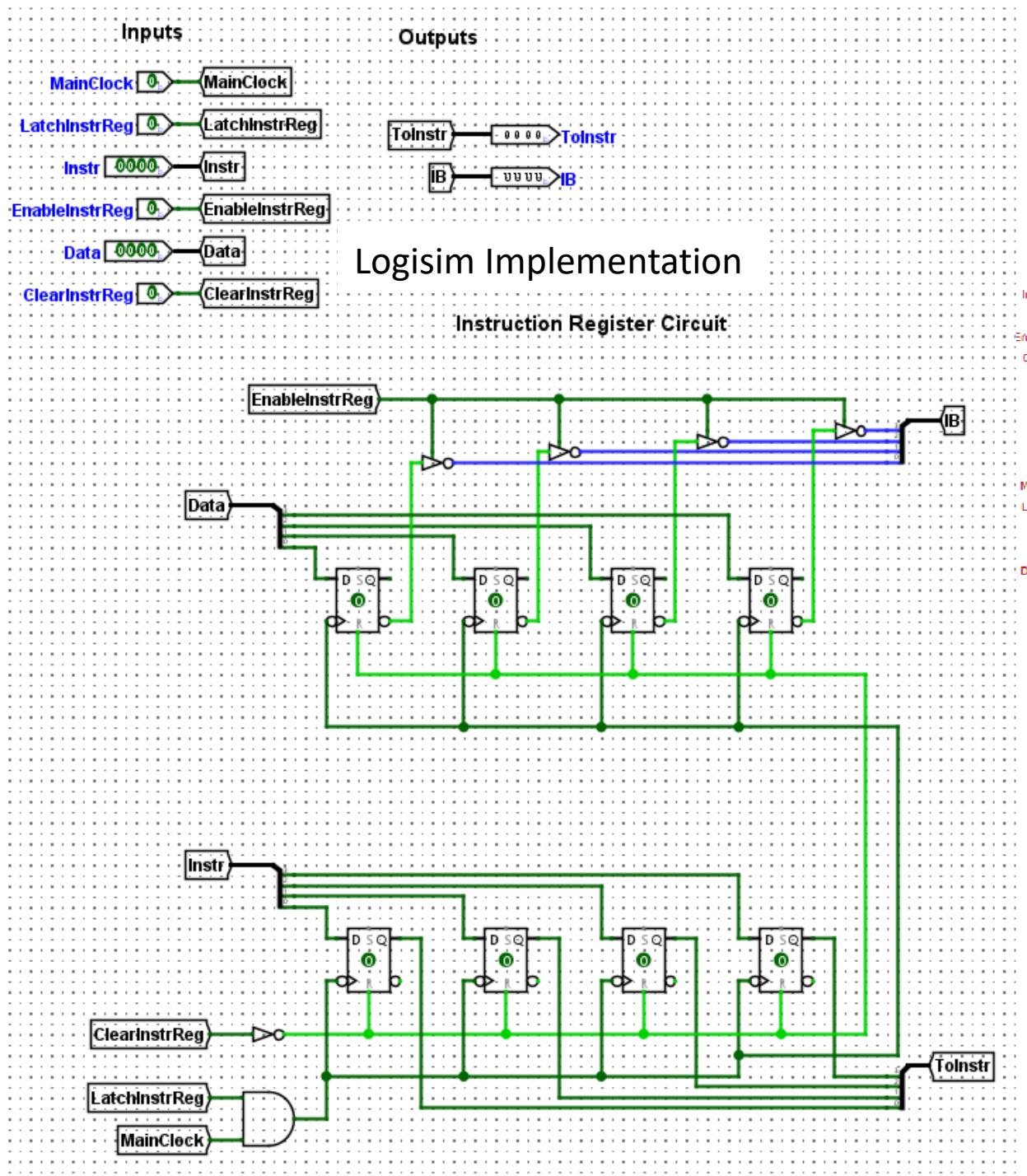


RTL View

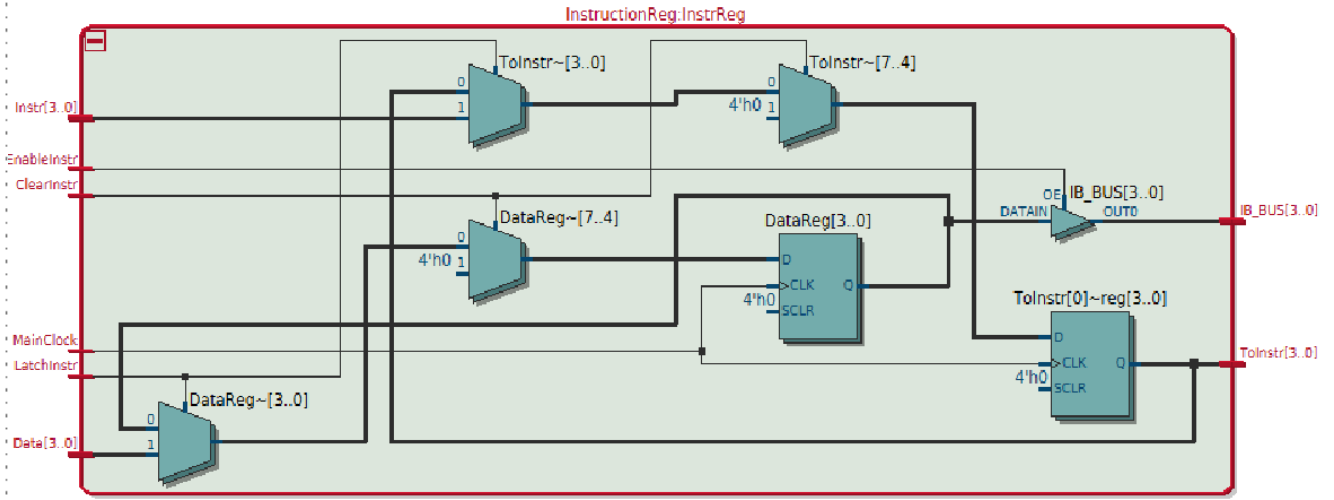


Instruction Register (InstructionReg.v)

- Stores the 8-bit instruction supplied by program memory.
- Separates each instruction into a 4-bit opcode and 4-bit operand.
- Captures both fields when LatchInstr is asserted.
- Sends the opcode to the FSM for instruction decoding.
- Places the stored operand on the internal bus when EnableInstr is asserted.
- Supports synchronous reset.


Inputs
Outputs
MainClock
LatchInstrReg
Instr
EnableInstrReg
Data
ClearInstrReg
ToInstr
IB
Logisim Implementation
Instruction Register Circuit
EnableInstrReg
IB
Data
Instr
ClearInstrReg
LatchInstrReg
MainClock
ToInstr

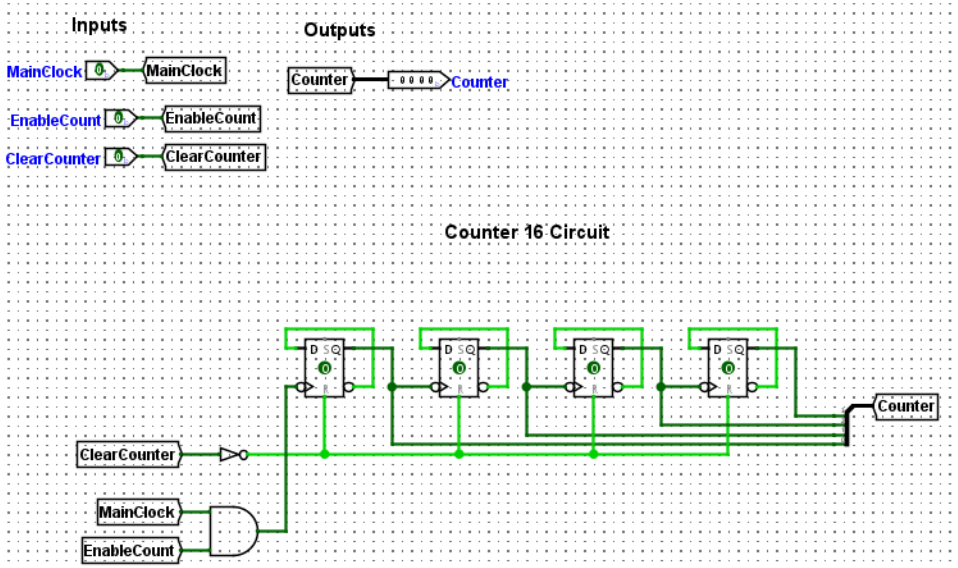
RTL View


InstructionReg:InstrReg
Instr[3..0]
EnableInstr
ClearInstr
MainClock
LatchInstr
Data[3..0]
ToInstr~[3..0]
ToInstr~[7..4]
4'h0
DataReg~[7..4]
DataReg~[3..0]
DataReg[3..0]
IB_BUS[3..0]
DATAIN
OUT0
OE
IB_BUS[3..0]
ToInstr[0]~reg[3..0]
ToInstr[3..0]
D
CLK
Q
SCLR

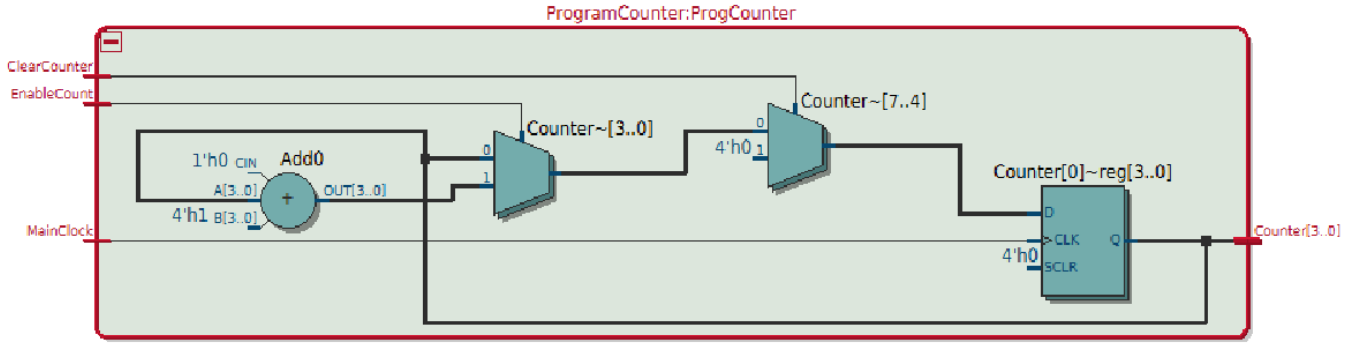
Program Counter(ProgramCounter.v)

- Maintains the current instruction address.
- Implements a 4-bit incrementing counter.
- Advances when EnableCount is asserted.
- Uses its lower three bits to address the eight-location ROM.

Logisim Implementation



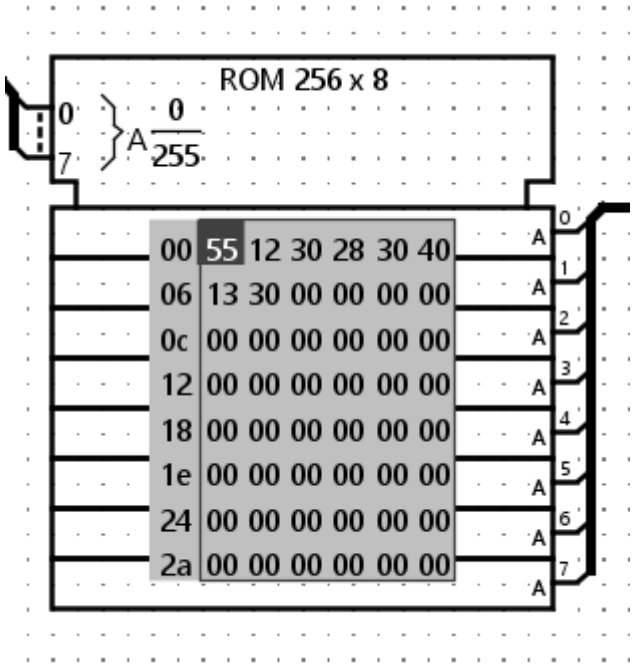
RTL View



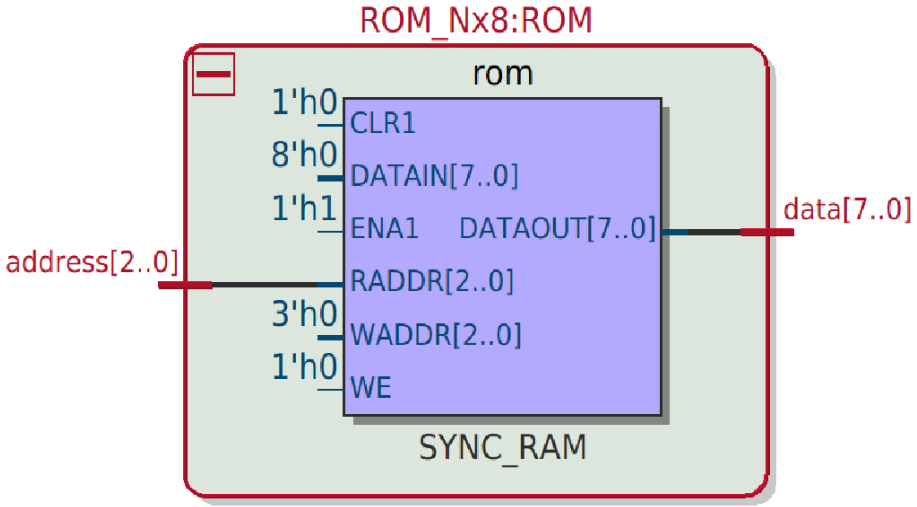
Program ROM (ROM_Nx8.v)

- Stores eight predefined 8-bit instructions.
- Supplies instructions according to the program counter address.
- Contains load, arithmetic, input, and output operations.
- Repeats the instruction sequence as the ROM address cycles.

Logisim Implementation

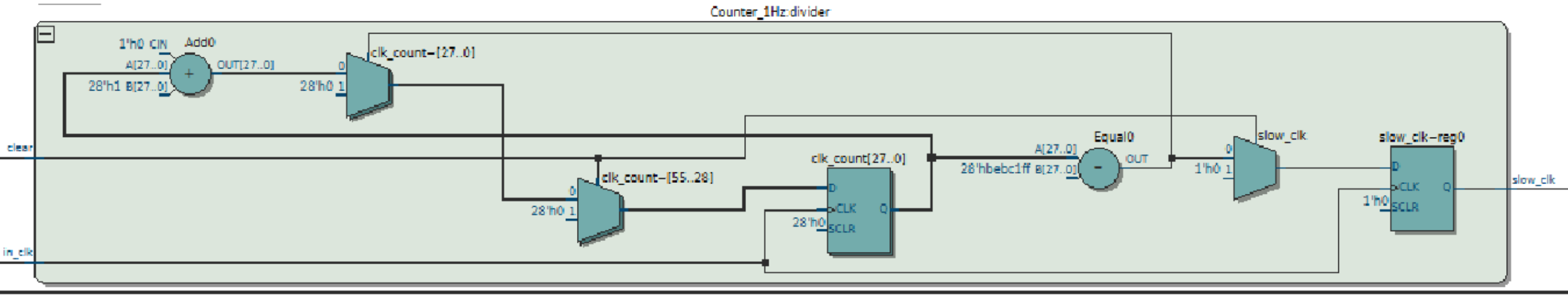


RTL View



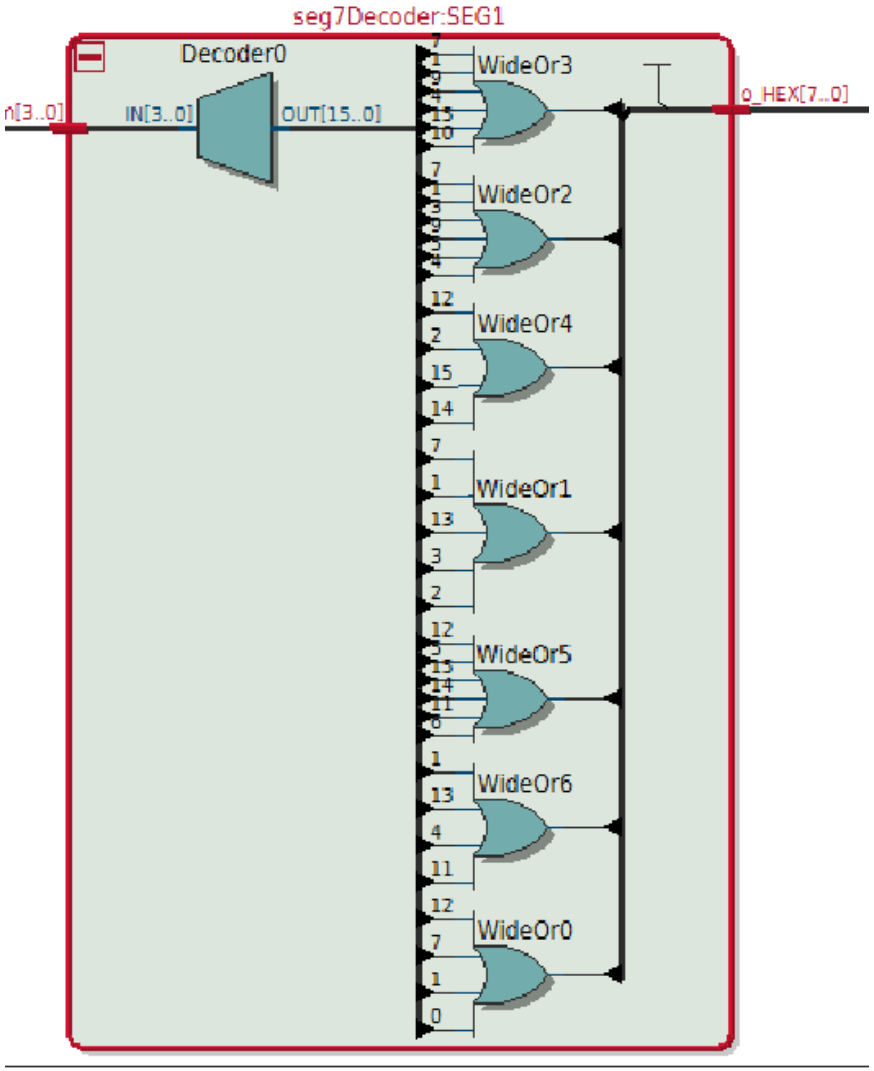
Clock Divider (Counter_1Hz.v)

- The Counter_1Hz module reduces the 50 MHz FPGA clock to a much slower timing signal used for instruction execution.



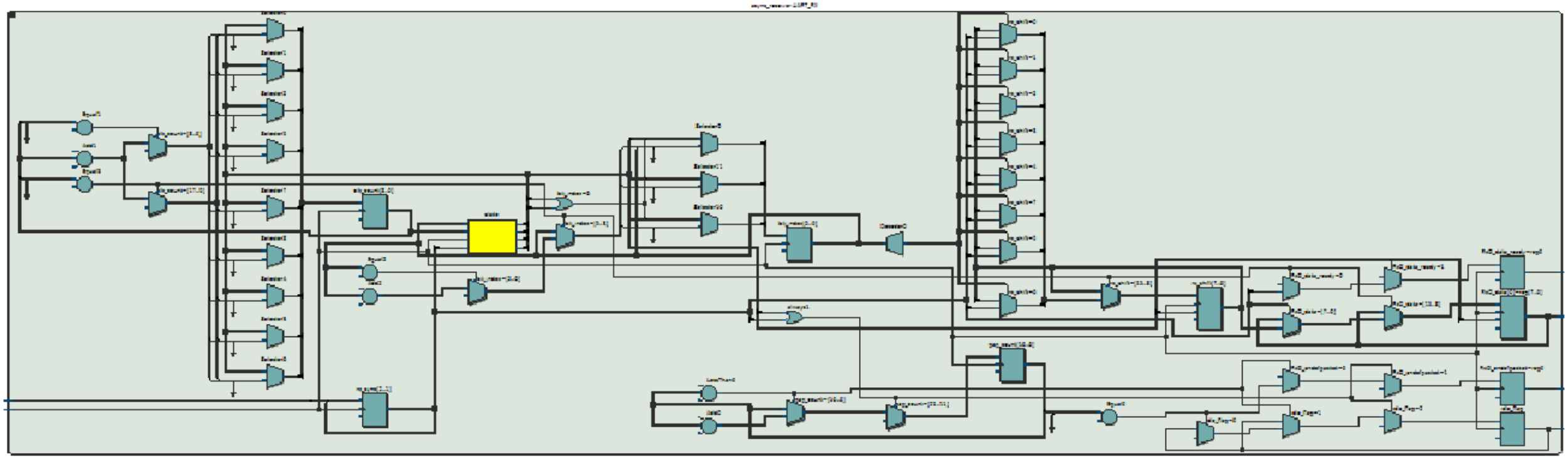
Seven-Segment Display Decoder (seg7Decoder.v)

- The seg7Decoder module converts a 4-bit binary value into the corresponding hexadecimal representation on a seven-segment display.



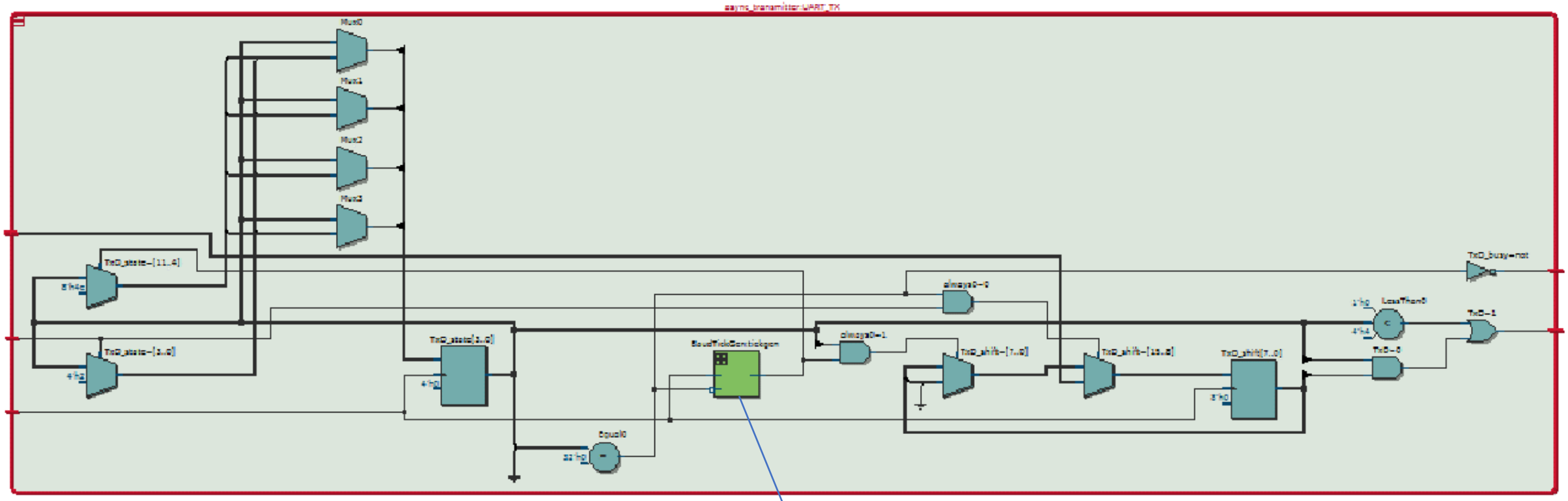
UART Receiver (async_receiver.v)

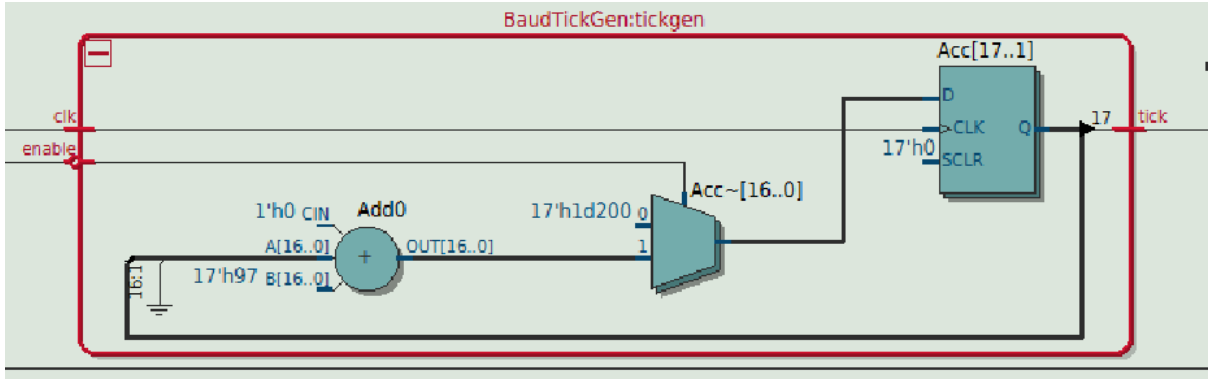
- The async_receiver module implements asynchronous serial data reception from an external device.



UART Transmitter (async_transmitter.v)

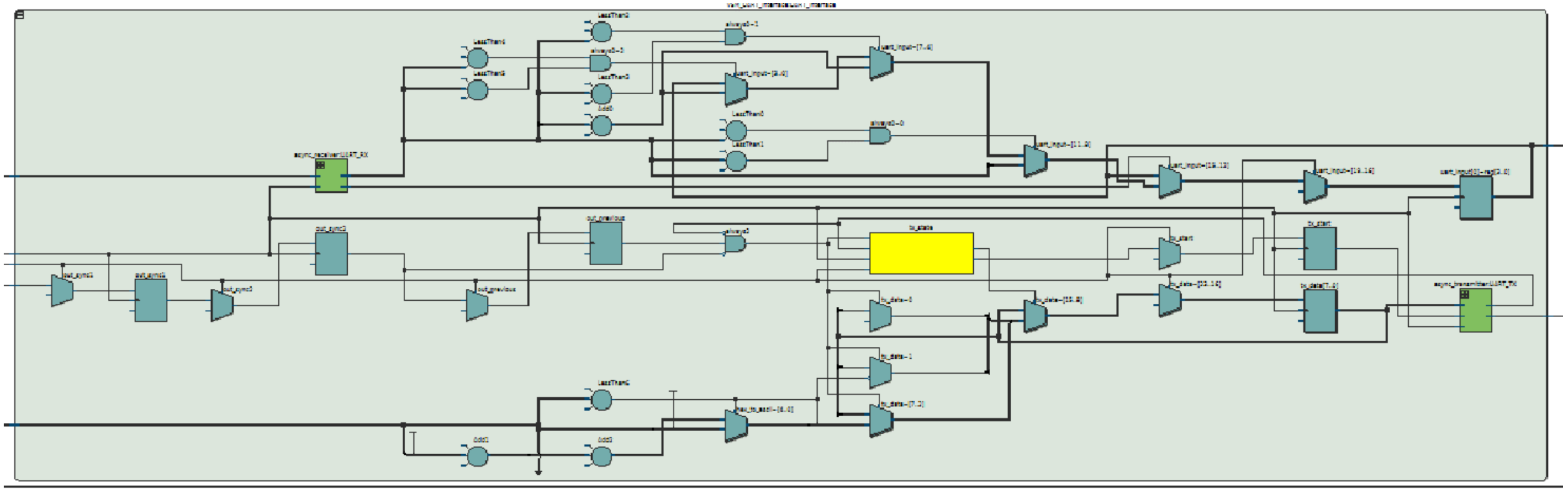
- The async_transmitter module converts an 8-bit parallel data value into a serial UART transmission.





VSM UART Interface (VSM_UART_Interface.v)

- The VSM_UART_Interface module provides the connection between the 4-bit VSM datapath and the 8-bit UART communication system.
- Its primary functions are serial data reception, ASCII-to-hexadecimal conversion, output-event detection, hexadecimal-to-ASCII conversion, and serial data transmission.



Input Selection and Hardware Interface

- SW[7] = 0: Selects the physical switches as processor input.
- SW[7] = 1: Selects the received UART input.
- SW[3:0]: Provides manual 4-bit input.
- SW[8]: Controls processor reset.
- GPIO[33]: UART receive connection.
- GPIO[35]: UART transmit connection.

Verilog HDL Source Code

4-Bit Very Simple Microprocessor (VSM)
with UART Communication

Terasic DE10-Lite FPGA — Intel MAX 10

Final Project — Source Code Appendix

November 7, 2026

Contents

Source Code Notes 2

1 Top-Level Integration 3

2 Microinstruction Control Unit 8

3 Accumulator A 12

4 Accumulator B 13

5 Arithmetic Unit 14

6 Input Register 15

7 Output Register 16

8 Instruction Register 17

9 Program Counter 18

10 Program ROM 19

11 Clock Divider 20

12 Seven-Segment Decoder 21

13 UART Receiver 22

14 UART Transmitter 26

15 Baud-Rate Generator 28

16 UART Parameter Check 29

17 VSM UART Interface 30

Source Code Notes

This document contains the 17 Verilog modules supplied for the VSM project. Each listing reproduces the corresponding uploaded source file. For modules with multiple similarly named files in the working directory, the uploaded numbered version is used as specified in the listing caption. The listings are provided for documentation and do not themselves establish successful synthesis or hardware verification.

1 Top-Level Integration

Source file: `main.v`

```
1
2  `default_nettype none
3
4  module main(
5      input MAX10_CLK1_50,
6      input [1:0] KEY,
7      input [9:0] SW,
8      inout [35:0] GPIO,
9      output [9:0] LEDR,
10     output [7:0] HEX0,
11     output [7:0] HEX1,
12     output [7:0] HEX2,
13     output [7:0] HEX4,
14     output [7:0] HEX5
15 );
16
17     localparam N = 4;
18
19     // ------------------------------------
20     // CLOCK AND RESET
21     // ------------------------------------
22
23     wire w_reset = SW[8];
24     wire w_slow_clock;
25     wire w_clock;
26
27     Counter_1Hz divider (
28         .in_clk(MAX10_CLK1_50),
29         .clear(w_reset),
30         .slow_clk(w_slow_clock)
31     );
32
33     // Provide a reset clock edge for the
34     // existing synchronous-reset VSM.
35     reg r_reset_clock = 1'b0;
36
37     always @(posedge MAX10_CLK1_50)
38         r_reset_clock <= w_reset;
39
40     assign w_clock = w_slow_clock | r_reset_clock;
41
42     // ------------------------------------
43     // UART GPIO CONNECTIONS
44     // ------------------------------------
45
46     wire w_uart_rx = GPIO[33];
47     wire w_uart_tx;
48
49     assign GPIO[35] = w_uart_tx;
50
51     // ------------------------------------
52     // UART INPUT SELECTION
53     // ------------------------------------
54
55     wire [N-1:0] w_user_input;
56     wire [N-1:0] w_uart_input;
57
```

```
    assign w_user_input =
        SW[7] ? w_uart_input : SW[3:0];

    // ------------------------------------
    // INTERNAL VSM WIRES
    // ------------------------------------

    wire [N-1:0] w_IB_BUS;
    wire [N-1:0] w_AluA;
    wire [N-1:0] w_AluB;

    wire [N-1:0] w_counter;
    wire [N-1:0] w_data;
    wire [N-1:0] w_instruction;
    wire [N-1:0] w_ToInstr;
    wire [N-1:0] w_rOut;

    wire w_carry;

    // ------------------------------------
    // FSM CONTROL WIRES
    // ------------------------------------

    wire w_LatchA;
    wire w_EnableA;
    wire w_LatchB;
    wire w_EnableALU;
    wire w_AddSub;
    wire w_EnableIN;
    wire w_EnableOut;
    wire w_LoadInstr;
    wire w_EnableInstr;
    wire w_EnableCount;

    // ------------------------------------
    // LED DEBUG OUTPUTS
    // ------------------------------------

    assign LEDR[9]   = w_carry;
    assign LEDR[8]   = SW[7];
    assign LEDR[7:4] = w_counter;
    assign LEDR[3:0] = w_rOut;

    // ------------------------------------
    // ACCUMULATOR A
    // ------------------------------------

    Accumulator_A AccA (
        .MainClock(w_clock),
        .ClearA(w_reset),
        .LatchA(w_LatchA),
        .EnableA(w_EnableA),
        .A(w_IB_BUS),
        .IB_BUS(w_IB_BUS),
        .AluA(w_AluA)
    );

    seg7Decoder SEG1 (
        .i_bin(w_AluA),
        .o_HEX(HEX1)
    );
```

```
119
120     // ------------------------------------
121     // ACCUMULATOR B
122     // ------------------------------------
123
124     Accumulator_B AccB (
125         .MainClock(w_clock),
126         .ClearB(w_reset),
127         .LatchB(w_LatchB),
128         .B(w_IB_BUS),
129         .AluB(w_AluB)
130     );
131
132     seg7Decoder SEG2 (
133         .i_bin(w_AluB),
134         .o_HEX(HEX2)
135     );
136
137     // ------------------------------------
138     // ARITHMETIC UNIT
139     // ------------------------------------
140
141     Arithmetic_Unit ALU (
142         .EnableALU(w_EnableALU),
143         .AddSub(w_AddSub),
144         .A(w_AluA),
145         .B(w_AluB),
146         .Carry(w_carry),
147         .IB_ALU(w_IB_BUS)
148     );
149
150     seg7Decoder SEG0 (
151         .i_bin(w_IB_BUS),
152         .o_HEX(HEX0)
153     );
154
155     // ------------------------------------
156     // INPUT REGISTER
157     // ------------------------------------
158
159     InRegister InReg (
160         .EnableIN(w_EnableIN),
161         .DataIn(w_user_input),
162         .IB_BUS(w_IB_BUS)
163     );
164
165     seg7Decoder SEG4 (
166         .i_bin(w_user_input),
167         .o_HEX(HEX4)
168     );
169
170     // ------------------------------------
171     // OUTPUT REGISTER
172     // ------------------------------------
173
174     OutRegister OutReg (
175         .MainClock(w_clock),
176         .MainReset(w_reset),
177         .EnableOut(w_EnableOut),
178         .IB_BUS(w_IB_BUS),
179         .rOut(w_rOut)
```

```
    );

    seg7Decoder SEG5 (
        .i_bin(w_rOut),
        .o_HEX(HEX5)
    );

    // ----------------------------------
    // INSTRUCTION REGISTER
    // ----------------------------------

    InstructionReg InstrReg (
        .MainClock(w_clock),
        .ClearInstr(w_reset),
        .LatchInstr(w_LoadInstr),
        .EnableInstr(w_EnableInstr),
        .Data(w_data),
        .Instr(w_instruction),
        .ToInstr(w_ToInstr),
        .IB_BUS(w_IB_BUS)
    );

    // ----------------------------------
    // PROGRAM COUNTER
    // ----------------------------------

    ProgramCounter ProgCounter (
        .MainClock(w_clock),
        .EnableCount(w_EnableCount),
        .ClearCounter(w_reset),
        .Counter(w_counter)
    );

    // ----------------------------------
    // PROGRAM ROM
    // ----------------------------------

    wire [7:0] w_rom_data;

    ROM_Nx8 ROM (
        .address(w_counter[2:0]),
        .data(w_rom_data)
    );

    assign {w_instruction, w_data} = w_rom_data;

    // ----------------------------------
    // MICROINSTRUCTION FSM
    // ----------------------------------

    FSM_MicroInstr Controller (
        .clk(w_clock),
        .reset(w_reset),
        .IB_BUS(w_IB_BUS),
        .LatchA(w_LatchA),
        .EnableA(w_EnableA),
        .LatchB(w_LatchB),
        .EnableALU(w_EnableALU),
        .AddSub(w_AddSub),
        .EnableIN(w_EnableIN),
        .EnableOut(w_EnableOut),
```

```
241         .LoadInstr(w_LoadInstr),
242         .EnableInstr(w_EnableInstr),
243         .ToInstr(w_ToInstr),
244         .EnableCount(w_EnableCount)
245     );
246 
247     // ------------------------------------
248     // UART INTERFACE
249     // ------------------------------------
250 
251     VSM_UART_Interface UART_Interface (
252         .clk(MAX10_CLK1_50),
253         .reset(w_reset),
254         .uart_rx(w_uart_rx),
255         .uart_tx(w_uart_tx),
256         .uart_input(w_uart_input),
257         .vsm_output(w_rOut),
258         .vsm_enable_out(w_EnableOut)
259     );
260 
261 endmodule
262 
263 `default_nettype wire
```

2 Microinstruction Control Unit

Source file: `FSM_MicroInstr(1).v`

```
1
2  `default_nettype none
3
4  module FSM_MicroInstr #
5  (
6      parameter N = 4
7  )
8  (
9      input clk,
10     input reset,
11     input [N-1:0] IB_BUS,
12
13     output reg LatchA,
14     output reg EnableA,
15     output reg LatchB,
16     output reg EnableALU,
17     output reg AddSub,
18     output reg EnableIN,
19     output reg EnableOut,
20     output reg LoadInstr,
21     output reg EnableInstr,
22     input [N-1:0]   ToInstr,
23     output reg EnableCount
24 );
25
26     reg [2:0] state, next_state;
27
28     // States
29     localparam [2:0] IDLE = 3'd0, PHASE_1 = 3'd1, PHASE_2 = 3'd2, PHASE_3 = 3'd3, PHASE_4 = 3'd4;
30
31     // 1. State Register (sequential)
32     always @(posedge clk) begin
33         if(reset)
34             state <= IDLE;
35         else
36             state <= next_state;
37     end
38
39     // 2. Next-State Logic (combinational)
40     always @(*) begin
41
42         next_state = state; // default: hold state
43
44         case(state)
45
46             IDLE:
47                 begin
48                     // Move to FETCH
49                     next_state = PHASE_1;
50                 end
51
52             // FETCH Instruction
53             PHASE_1:
54                 begin
55                     // Move to DECODE
56                     next_state = PHASE_2;
57                 end
```

```
58
59              // DECODE Instruction
60              PHASE_2:
61                  begin
62                      // Move to EXECUTE A
63                      next_state = PHASE_3;
64                  end
65
66              // EXECUTE A Instruction
67              PHASE_3:
68                  begin
69                      // Move to EXECUTE B
70                      next_state = PHASE_4;
71                  end
72
73              // EXECUTE B Instruction
74              PHASE_4:
75                  begin
76                      // Move to FETCH
77                      next_state = PHASE_1;
78                  end
79
80              default: ;// None
81
82          endcase
83      end
84
85      // 3. Output Logic (combinational)
86
87      always @(*) begin
88
89          // default all signals are zero
90          LoadInstr   = 1'b0;
91          EnableInstr = 1'b0;
92          LatchB      = 1'b0;
93          LatchA      = 1'b0;
94          EnableALU   = 1'b0;
95          EnableCount = 1'b0;
96          AddSub      = 1'b0;
97          EnableIN        = 1'b0;
98          EnableA     = 1'b0;
99          EnableOut   = 1'b0;
100
101         case(state)
102
103             IDLE:
104                 begin
105                     // Initialize things if needed
106                 end
107
108             // FETCH Instruction
109             PHASE_1:
110                 begin
111                     // same sequence for all asm instructions
112                     LoadInstr = 1'b1;
113                 end
114
115             // DECODE Instruction
116             PHASE_2:
117                 begin
118                     // same sequence for all asm instructions
```

```
119                     EnableCount = 1'b1;
120                     EnableInstr = 1'b1;
121                 end
122 
123             // EXECUTE Instruction
124             PHASE_3:
125                 begin
126 
127                     case(ToInstr)
128 
129                         4'b0000: // NOP
130                             begin
131                                 // do nothing
132                             end
133 
134                         4'b0001: // ADD
135                             begin
136                                 LatchB      = 1'b1;
137                                 EnableInstr = 1'b1;
138                             end
139 
140                         4'b0010: // SUB
141                             begin
142                                 LatchB      = 1'b1;
143                                 EnableInstr = 1'b1;
144                             end
145 
146                         4'b0011: // OUT
147                             begin
148                                 EnableOut   = 1'b1;
149                                 EnableA         = 1'b1;
150                             end
151 
152                         4'b0100: // IN
153                             begin
154                                 LatchA  = 1'b1;
155                                 EnableIN = 1'b1;
156                             end
157 
158                         4'b0101: // LDA
159                             begin
160                                 LatchA      = 1'b1;
161                                 EnableInstr = 1'b1;
162                             end
163 
164                     endcase
165 
166                 end
167 
168             // EXECUTE Instruction
169             PHASE_4:
170                 begin
171 
172                     case(ToInstr)
173 
174                         4'b0000: // NOP
175                             begin
176                                 // do nothing
177                             end
178 
179                         4'b0001: // ADD
```

```
180                             begin
181                                 LatchA      = 1'b1;
182                                 EnableALU   = 1'b1;
183                             end
184
185                         4'b0010: // SUB
186                             begin
187                                 LatchA      = 1'b1;
188                                 EnableALU   = 1'b1;
189                                 AddSub      = 1'b1;
190                             end
191
192                         4'b0011: // OUT
193                             begin
194                                 // do nothing
195                             end
196
197                         4'b0100: // IN
198                             begin
199                                 // do nothing
200                             end
201
202                         4'b0101: // LDA
203                             begin
204                                 // do nothing
205                             end
206
207                     endcase
208
209                 end
210
211         endcase
212     end
213
214 endmodule
215
216 `default_nettype wire
```

3 Accumulator A

Source file: `Accumulator_A(1).v`

```
1
2  `default_nettype none
3
4  module Accumulator_A #
5  (
6      parameter N = 4
7  )
8  (
9      input MainClock,
10     input ClearA,
11     input LatchA,
12     input EnableA,
13     input [N-1:0] A,
14     output [N-1:0] IB_BUS,
15     output reg [N-1:0] AluA
16 );
17
18     // Sequential Logic
19     always @(posedge MainClock) begin
20         if(ClearA)
21             AluA <= {N{1'b0}};
22         else begin
23             if(LatchA)
24                 AluA <= A;
25         end
26     end
27
28     // Combinational Logic
29     assign IB_BUS = (EnableA) ? AluA : {N{1'bz}};
30
31 endmodule
32
33 `default_nettype wire
```

4 Accumulator B

Source file: `Accumulator_B.v`

```
1
2  `default_nettype none
3
4  module Accumulator_B #
5  (
6      parameter N = 4
7  )
8  (
9      input MainClock,
10     input ClearB,
11     input LatchB,
12     input [N-1:0] B,
13     output reg [N-1:0] AluB
14 );
15
16     // Sequential Logic
17     always @(posedge MainClock) begin
18         if(ClearB)
19             AluB <= {N{1'b0}};
20         else begin
21             if(LatchB)
22                 AluB <= B;
23         end
24     end
25
26
27 endmodule
28
29 `default_nettype wire
```

5 Arithmetic Unit

Source file: `Arithmetic_Unit.v`

```
1
2  `default_nettype none
3
4  module Arithmetic_Unit #
5  (
6      parameter N = 4
7  )
8  (
9      input EnableALU,
10     input AddSub,
11     input [N-1:0] A,
12     input [N-1:0] B,
13     output wire Carry,
14     output wire [N-1:0] IB_ALU
15 );
16
17     wire [N:0] sum;
18
19     // AddSub = 0: Addition (A + B)
20     // AddSub = 1: Subtraction (B - A)
21
22     assign sum = AddSub ?
23                  ({1'b0, B} - {1'b0, A}) :
24                  ({1'b0, A} + {1'b0, B});
25
26     assign Carry = sum[N];
27
28     assign IB_ALU = EnableALU ?
29                     sum[N-1:0] : {N{1'bz}};
30
31 endmodule
32
33 `default_nettype wire
```

6 Input Register

Source file: `InRegister.v`

```
1
2  `default_nettype none
3
4  module InRegister #
5  (
6      parameter N = 4
7  )
8  (
9      input EnableIN,
10     input  [N-1:0] DataIn,
11     output [N-1:0] IB_BUS
12 );
13
14     assign IB_BUS = (EnableIN) ? DataIn : {N{1'bz}};
15
16 endmodule
17
18 `default_nettype wire
```

7 Output Register

Source file: `OutRegister.v`

```
1
2  `default_nettype none
3
4  module OutRegister #
5  (
6      parameter N = 4
7  )
8  (
9      input MainClock,
10     input MainReset,
11     input EnableOut,
12     input [N-1:0] IB_BUS,
13     output reg [N-1:0] rOut
14 );
15
16     always @(posedge MainClock)
17     begin
18         if (MainReset)
19         begin
20             rOut <= {N{1'b0}};
21         end
22         else if (EnableOut)
23         begin
24             rOut <= IB_BUS;
25         end
26     end
27
28 endmodule
29
30 `default_nettype wire
```

8 Instruction Register

Source file: `InstructionReg.v`

```
1
2  `default_nettype none
3
4  module InstructionReg #
5  (
6      parameter N = 4
7  )
8  (
9      input MainClock,
10     input ClearInstr,
11     input LatchInstr,
12     input EnableInstr,
13     input [N-1:0] Data,
14     input [N-1:0] Instr,
15     output reg [N-1:0] ToInstr,
16     output [N-1:0] IB_BUS
17 );
18
19     reg [N-1:0] DataReg;
20
21     always @(posedge MainClock)
22     begin
23         if (ClearInstr)
24         begin
25             ToInstr <= {N{1'b0}};
26             DataReg <= {N{1'b0}};
27         end
28         else if (LatchInstr)
29         begin
30             ToInstr <= Instr;
31             DataReg <= Data;
32         end
33     end
34
35     assign IB_BUS = EnableInstr ?
36                     DataReg : {N{1'bz}};
37
38 endmodule
39
40 `default_nettype wire
```

9 Program Counter

Source file: `ProgramCounter.v`

```
1
2  `default_nettype none
3
4  module ProgramCounter #
5  (
6      parameter N = 4
7  )
8  (
9      input MainClock,
10     input EnableCount,
11     input ClearCounter,
12     output reg [N-1:0] Counter
13 );
14
15     always @(posedge MainClock)
16     begin
17         if (ClearCounter)
18         begin
19             Counter <= {N{1'b0}};
20         end
21         else if (EnableCount)
22         begin
23             Counter <= Counter + 1'b1;
24         end
25     end
26
27 endmodule
28
29 `default_nettype wire
```

10 Program ROM

Source file: `ROM_Nx8(1).v`

```
1
2  `default_nettype none
3
4  module ROM_Nx8 #
5  (
6      parameter N = 8
7  )
8  (
9      input [$clog2(N)-1:0] address,
10     output reg [7:0] data
11 );
12
13     (* ramstyle = "logic" *) reg [7:0] rom [0:N-1];
14
15     // Initialize memory -> your asm instructions
16     initial begin
17         rom[0] = 8'h55;
18         rom[1] = 8'h12;
19         rom[2] = 8'h30;
20         rom[3] = 8'h28;
21         rom[4] = 8'h30;
22         rom[5] = 8'h40;
23         rom[6] = 8'h13;
24         rom[7] = 8'h30;
25         // maximum 16 instructions
26     end
27
28     // Combinational Logic (async)
29     always @(*) begin
30         data = rom[address];
31     end
32
33 endmodule
34
35 `default_nettype wire
```

11 Clock Divider

Source file: `Counter_1Hz(1).v`

```
1
2  module Counter_1Hz(
3      input in_clk,
4      input clear,
5      output reg slow_clk
6  );
7
8      reg [27:0] clk_count;
9
10     // Clock divider: 50 MHz --> One pulse every 4 seconds
11     // Synchronous reset
12
13     always @(posedge in_clk) begin
14         if (clear) begin
15             slow_clk <= 0;
16             clk_count <= 28'd0;
17         end
18         else begin
19             if (clk_count == 28'd199_999_999) begin
20                 clk_count <= 28'd0;
21                 slow_clk <= 1;
22             end
23             else begin
24                 clk_count <= clk_count + 1'b1;
25                 slow_clk <= 0;
26             end
27         end
28     end
29
30 endmodule
```

12 Seven-Segment Decoder

Source file: `seg7Decoder.v`

```
1
2  `default_nettype none
3
4  module seg7Decoder (
5      input [3:0] i_bin,
6      output reg [7:0] o_HEX
7  );
8
9      always @(*) begin
10
11         // required to not infer latches
12         o_HEX = 8'b11111111;
13
14         case(i_bin)
15             4'd0: o_HEX = 8'b11000000;
16             4'd1: o_HEX = 8'b11111001;
17             4'd2: o_HEX = 8'b10100100;
18             4'd3: o_HEX = 8'b10110000;
19             4'd4: o_HEX = 8'b10011001;
20             4'd5: o_HEX = 8'b10010010;
21             4'd6: o_HEX = 8'b10000010;
22             4'd7: o_HEX = 8'b11111000;
23             4'd8: o_HEX = 8'b10000000;
24             4'd9: o_HEX = 8'b10011000;
25             4'd10: o_HEX = 8'b10001000;
26             4'd11: o_HEX = 8'b10000011;
27             4'd12: o_HEX = 8'b11000110;
28             4'd13: o_HEX = 8'b10100001;
29             4'd14: o_HEX = 8'b10000110;
30             4'd15: o_HEX = 8'b10001110;
31
32         endcase
33
34     end
35
36 endmodule
37
38 `default_nettype wire
```

13 UART Receiver

Source file: `async_receiver.v`

```
1
2  `default_nettype none
3
4  module async_receiver #(
5      parameter ClkFrequency = 50000000,
6      parameter Baud = 115200,
7      parameter Oversampling = 8
8  )(
9      input clk,
10     input RxD,
11
12     output reg RxD_data_ready = 1'b0,
13     output reg [7:0] RxD_data = 8'd0,
14
15     output wire RxD_idle,
16     output reg RxD_endofpacket = 1'b0
17 );
18
19     // ---------------------------------------
20     // UART TIMING
21     // ---------------------------------------
22
23     localparam integer CLKS_PER_BIT =
24         ClkFrequency / Baud;
25
26     localparam integer HALF_BIT =
27         CLKS_PER_BIT / 2;
28
29     localparam integer COUNT_WIDTH =
30         $clog2(CLKS_PER_BIT + 1);
31
32     // ---------------------------------------
33     // UART STATES
34     // ---------------------------------------
35
36     localparam [1:0]
37         IDLE  = 2'd0,
38         START = 2'd1,
39         DATA  = 2'd2,
40         STOP  = 2'd3;
41
42     reg [1:0] state = IDLE;
43
44     // ---------------------------------------
45     // SYNCHRONIZE RX INPUT
46     // ---------------------------------------
47
48     reg rx_sync1 = 1'b1;
49     reg rx_sync2 = 1'b1;
50
51     always @(posedge clk) begin
52         rx_sync1 <= RxD;
53         rx_sync2 <= rx_sync1;
54     end
55
56     // ---------------------------------------
57     // INTERNAL REGISTERS
```

```
58      // ----------------------------------------
59
60      reg [COUNT_WIDTH-1:0] clk_count = 0;
61
62      reg [2:0] bit_index = 0;
63
64      reg [7:0] rx_shift = 0;
65
66      // ----------------------------------------
67      // IDLE / END OF PACKET DETECTION
68      // ----------------------------------------
69
70      localparam integer GAP_BITS = 4;
71
72      localparam integer GAP_CLKS =
73          CLKS_PER_BIT * GAP_BITS;
74
75      localparam integer GAP_WIDTH =
76          $clog2(GAP_CLKS + 1);
77
78      reg [GAP_WIDTH-1:0] gap_count = 0;
79      reg idle_flag = 1'b0;
80
81      assign RxD_idle = idle_flag;
82
83      always @(posedge clk) begin
84
85          RxD_endofpacket <= 1'b0;
86
87          if (state != IDLE || rx_sync2 == 1'b0) begin
88              gap_count <= 0;
89              idle_flag <= 1'b0;
90          end
91
92          else if (gap_count < GAP_CLKS) begin
93              gap_count <= gap_count + 1'b1;
94
95              if (gap_count == GAP_CLKS - 1) begin
96                  idle_flag <= 1'b1;
97                  RxD_endofpacket <= 1'b1;
98              end
99          end
100
101     end
102
103     // ----------------------------------------
104     // UART RECEIVER STATE MACHINE
105     // ----------------------------------------
106
107     always @(posedge clk) begin
108
109         // Default: data-ready is a one-clock pulse
110         RxD_data_ready <= 1'b0;
111
112         case (state)
113
114             // ------------------------------
115             // WAIT FOR START BIT
116             // ------------------------------
117
118             IDLE: begin
```

```
119
120                 clk_count <= 0;
121                 bit_index <= 0;
122
123                 if (rx_sync2 == 1'b0)
124                     state <= START;
125
126             end
127
128             // -------------------------------
129             // VERIFY START BIT
130             // -------------------------------
131
132             START: begin
133
134                 if (clk_count == HALF_BIT - 1) begin
135
136                     clk_count <= 0;
137
138                     if (rx_sync2 == 1'b0)
139                         state <= DATA;
140                     else
141                         state <= IDLE;
142
143                 end
144
145                 else begin
146                     clk_count <= clk_count + 1'b1;
147                 end
148
149             end
150
151             // -------------------------------
152             // RECEIVE 8 DATA BITS
153             // -------------------------------
154
155             DATA: begin
156
157                 if (clk_count == CLKS_PER_BIT - 1) begin
158
159                     clk_count <= 0;
160
161                     rx_shift[bit_index] <= rx_sync2;
162
163                     if (bit_index == 3'd7) begin
164                         bit_index <= 0;
165                         state <= STOP;
166                     end
167
168                     else begin
169                         bit_index <= bit_index + 1'b1;
170                     end
171
172                 end
173
174                 else begin
175                     clk_count <= clk_count + 1'b1;
176                 end
177
178             end
179
```

```
            // -------------------------------
            // VERIFY STOP BIT
            // -------------------------------

            STOP: begin

                if (clk_count == CLKS_PER_BIT - 1) begin

                    clk_count <= 0;
                    state <= IDLE;

                    if (rx_sync2 == 1'b1) begin

                        RxD_data <= rx_shift;

                        RxD_data_ready <= 1'b1;

                    end

                end

                else begin
                    clk_count <= clk_count + 1'b1;
                end

            end

            default: begin

                state <= IDLE;
                clk_count <= 0;

            end

        endcase

    end

endmodule

`default_nettype wire
```

14 UART Transmitter

Source file: `async_transmitter.v`

```
1
2  module async_transmitter(
3      input clk,
4      input TxD_start,
5      input [7:0] TxD_data,
6      output TxD,
7      output TxD_busy
8  );
9
10 parameter ClkFrequency = 50000000;
11 parameter Baud = 115200;
12
13 generate
14     if (ClkFrequency < Baud*8 &&
15         (ClkFrequency % Baud != 0))
16         ASSERTION_ERROR PARAMETER_OUT_OF_RANGE();
17 endgenerate
18
19 `ifdef SIMULATION
20 wire BitTick = 1'b1;
21 `else
22 wire BitTick;
23
24 BaudTickGen #(
25     .ClkFrequency(ClkFrequency),
26     .Baud(Baud)
27 ) tickgen (
28     .clk(clk),
29     .enable(TxD_busy),
30     .tick(BitTick)
31 );
32 `endif
33
34 reg [3:0] TxD_state = 0;
35
36 wire TxD_ready = (TxD_state == 0);
37
38 assign TxD_busy = ~TxD_ready;
39
40 reg [7:0] TxD_shift = 0;
41
42 always @(posedge clk)
43 begin
44
45     if (TxD_ready && TxD_start)
46         TxD_shift <= TxD_data;
47
48     else if (TxD_state[3] && BitTick)
49         TxD_shift <= TxD_shift >> 1;
50
51     case (TxD_state)
52
53         4'b0000:
54             if (TxD_start)
55                 TxD_state <= 4'b0100;
56
57         4'b0100:
```

```
58              if (BitTick)
59                  TxD_state <= 4'b1000;
60
61          4'b1000:
62              if (BitTick)
63                  TxD_state <= 4'b1001;
64
65          4'b1001:
66              if (BitTick)
67                  TxD_state <= 4'b1010;
68
69          4'b1010:
70              if (BitTick)
71                  TxD_state <= 4'b1011;
72
73          4'b1011:
74              if (BitTick)
75                  TxD_state <= 4'b1100;
76
77          4'b1100:
78              if (BitTick)
79                  TxD_state <= 4'b1101;
80
81          4'b1101:
82              if (BitTick)
83                  TxD_state <= 4'b1110;
84
85          4'b1110:
86              if (BitTick)
87                  TxD_state <= 4'b1111;
88
89          4'b1111:
90              if (BitTick)
91                  TxD_state <= 4'b0010;
92
93          4'b0010:
94              if (BitTick)
95                  TxD_state <= 4'b0011;
96
97          4'b0011:
98              if (BitTick)
99                  TxD_state <= 4'b0000;
100
101         default:
102             if (BitTick)
103                 TxD_state <= 4'b0000;
104
105     endcase
106 end
107
108 assign TxD =
109     (TxD_state < 4) |
110     (TxD_state[3] & TxD_shift[0]);
111
112 endmodule
```

15 Baud-Rate Generator

Source file: `BaudTickGen.v`

```
1
2  module BaudTickGen(
3      input clk,
4      input enable,
5      output tick
6  );
7
8  parameter ClkFrequency = 50000000;
9  parameter Baud = 115200;
10 parameter Oversampling = 1;
11
12 function integer log2;
13     input integer v;
14     begin
15         log2 = 0;
16         while (v >> log2)
17             log2 = log2 + 1;
18     end
19 endfunction
20
21 localparam AccWidth =
22     log2(ClkFrequency/Baud) + 8;
23
24 reg [AccWidth:0] Acc = 0;
25
26 localparam ShiftLimiter =
27     log2(Baud*Oversampling >> (31-AccWidth));
28
29 localparam Inc =
30     ((Baud*Oversampling <<
31       (AccWidth-ShiftLimiter)) +
32      (ClkFrequency >> (ShiftLimiter+1))) /
33     (ClkFrequency >> ShiftLimiter);
34
35 always @(posedge clk)
36 begin
37     if (enable)
38         Acc <= Acc[AccWidth-1:0] +
39                Inc[AccWidth:0];
40     else
41         Acc <= Inc[AccWidth:0];
42 end
43
44 assign tick = Acc[AccWidth];
45
46 endmodule
```

16 UART Parameter Check

Source file: `ASSERTION_ERROR.v`

```
1
2 module ASSERTION_ERROR();
3 endmodule
```

17 VSM UART Interface

Source file: `VSM_UART_Interface.v`

```
`default_nettype none

module VSM_UART_Interface (
    input clk,
    input reset,

    input uart_rx,
    output uart_tx,

    output reg [3:0] uart_input,

    input [3:0] vsm_output,
    input vsm_enable_out
);

    // ------------------------------------
    // UART RECEIVER
    // ------------------------------------

    wire [7:0] rx_data;
    wire rx_ready;
    wire rx_idle;
    wire rx_endofpacket;

    async_receiver #(
        .ClkFrequency(50000000),
        .Baud(115200)
    ) UART_RX (
        .clk(clk),
        .RxD(uart_rx),
        .RxD_data_ready(rx_ready),
        .RxD_data(rx_data),
        .RxD_idle(rx_idle),
        .RxD_endofpacket(rx_endofpacket)
    );

    // ------------------------------------
    // ASCII HEX TO 4-BIT DATA
    // ------------------------------------

    initial uart_input = 4'd0;

    always @(posedge clk) begin
        if (reset)
            uart_input <= 4'd0;
        else if (rx_ready) begin
            if (rx_data >= 8'h30 &&
                rx_data <= 8'h39)
                uart_input <= rx_data[3:0];

            else if (rx_data >= 8'h41 &&
                     rx_data <= 8'h46)
                uart_input <= rx_data[3:0] + 4'd9;

            else if (rx_data >= 8'h61 &&
                     rx_data <= 8'h66)
                uart_input <= rx_data[3:0] + 4'd9;
```

```
        end
    end

    // ----------------------------------
    // UART TRANSMITTER
    // ----------------------------------

    reg tx_start = 1'b0;
    reg [7:0] tx_data = 8'h30;
    wire tx_busy;

    async_transmitter #(
        .ClkFrequency(50000000),
        .Baud(115200)
    ) UART_TX (
        .clk(clk),
        .TxD_start(tx_start),
        .TxD_data(tx_data),
        .TxD(uart_tx),
        .TxD_busy(tx_busy)
    );

    // ----------------------------------
    // SYNCHRONIZE OUTPUT CONTROL
    // ----------------------------------

    reg out_sync1 = 1'b0;
    reg out_sync2 = 1'b0;
    reg out_previous = 1'b0;

    always @(posedge clk) begin
        if (reset) begin
            out_sync1 <= 1'b0;
            out_sync2 <= 1'b0;
            out_previous <= 1'b0;
        end
        else begin
            out_sync1 <= vsm_enable_out;
            out_sync2 <= out_sync1;
            out_previous <= out_sync2;
        end
    end

    // Falling edge indicates the OUT
    // register load has completed.
    wire out_complete =
        out_previous && !out_sync2;

    // ----------------------------------
    // HEX TO ASCII CONVERSION
    // ----------------------------------

    function [7:0] hex_to_ascii;
        input [3:0] value;
        begin
            if (value < 4'd10)
                hex_to_ascii = 8'h30 + {4'b0, value};
            else
                hex_to_ascii = 8'h41 +
                               {4'b0, (value - 4'd10)};
        end
```

```
    endfunction

    // ------------------------------------
    // TRANSMIT CONTROL
    // Send one character for each OUT.
    // ------------------------------------

    localparam TX_IDLE = 2'd0;
    localparam TX_START = 2'd1;
    localparam TX_WAIT = 2'd2;

    reg [1:0] tx_state = TX_IDLE;

    always @(posedge clk) begin
        if (reset) begin
            tx_start <= 1'b0;
            tx_data <= 8'h30;
            tx_state <= TX_IDLE;
        end
        else begin
            tx_start <= 1'b0;

            case (tx_state)

                TX_IDLE: begin
                    if (out_complete && !tx_busy) begin
                        tx_data <= hex_to_ascii(vsm_output);
                        tx_state <= TX_START;
                    end
                end

                TX_START: begin
                    tx_start <= 1'b1;
                    tx_state <= TX_WAIT;
                end

                TX_WAIT: begin
                    if (tx_busy)
                        tx_state <= TX_IDLE;
                end

                default:
                    tx_state <= TX_IDLE;

            endcase
        end
    end

endmodule

`default_nettype wire
```